

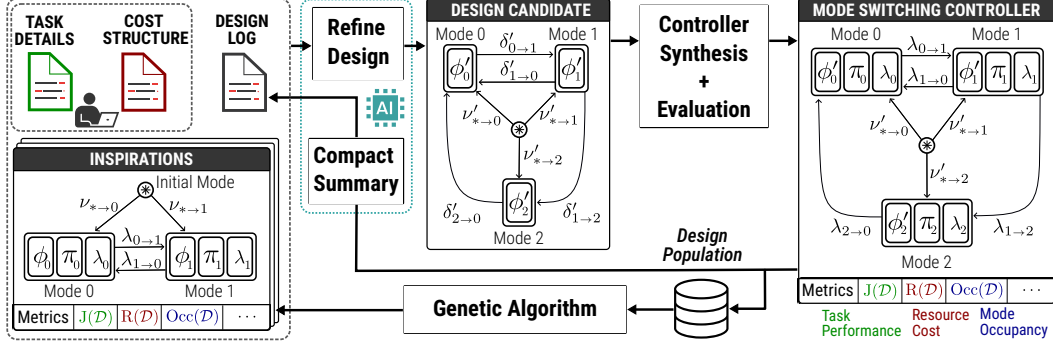


Figure 2: LEMCA System Diagram: An LLM-in-the loop evolutionary search pipeline to synthesize and refine mode-switching control architectures represented as a finite state machine specifying various operating modes (containing the sensory-compute configuration ϕ_j , control policy π_j , and monitor policy λ_j), and mode-transitions δ (see Sec. 2). A controller-synthesis procedure is used to generate a performant controller adhering to the design specified. The resulting controller’s performance metrics serve as feedback for iterative refinement.

more holistically include computation, energy, latency, etc. Further, we often seek not one solution but a “Pareto frontier” family of solutions that optimally trade off task performance for resource costs. Finally, most real-world tasks like driving and object manipulation naturally have dramatically time-varying resource needs [6, 7]: navigating a cluttered hallway with other dynamic agents may require high-frequency LiDAR and careful path planning, while the same robot traversing a clear hallway could operate nominally with minimal sensing using simple wall-following heuristics. Indeed, we will see in our experiments that even the point-to-disk task has time-varying resource needs.

In practice, we therefore resort to making educated guesses when allocating resources for a robotic control policy. When we have sufficient expertise in a task domain, we might attempt to design “mode-switching controllers” (MSCs), that dynamically switch between manually specified task modes with guesstimated design decisions including resource allocations [8, 9, 10]. However, even this is out of fashion in general-purpose robotics where per-task laborious expert-driven controller synthesis is prohibitive. Instead, we often fall back to a wasteful “monolithic” one-size-fits-all control architecture that implicitly anchors all resource allocations to the hardest, most resource-intensive task phases: our navigation robot above might run high-frequency LiDAR sensing and overly comprehensive path planning computations even in empty hallways.

We propose LEMCA, LLM-Guided synthesis of Efficient Mode-Switching Control Architectures, an automated approach to synthesize MSC designs for new tasks in simulation (see Figure 2). This involves the following key contributions:

- **MSC Design Representation.** We represent MSC designs as programs that specify controller modes, sensory-compute resource allocations, and state-dependent mode transition functions.
- **Problem Formulation as Program Synthesis.** This permits formulating MSC design as a program synthesis problem, opening up new tools for automating MSC design.
- **LLM-Guided Design Sampling.** We propose a task-generic LLM-guided evolutionary search pipeline, LEMCA to produce performant-yet-efficient MSC designs. This brings LLM coding and semantic reasoning capabilities to bear on exploring the large MSC design space.
- **MSC-Compatible Reinforcement Learning.** We propose a generic reinforcement learning paradigm to learn controllers subject to an MSC design specification.
- **Empirical Task Suite and Validation.** We design a diverse set of evaluation tasks for MSC design. Besides variations of the point-to-disk task above, this includes rangefinder variants of `dm_control` tasks [11] and planar navigation.

Our experimental results establish that LEMCA consistently discovers MSC designs that push beyond the Pareto frontier of monolithic designs.

2 LLM-Guided synthesis of Efficient Mode-Switching Control Architectures

Consider a task specified by a Markov Decision Process (MDP) $\mathcal{M} := (\mathcal{S}, \mathcal{A}, \mathcal{R}, \mathcal{P}, \mu)$, with state space $\mathcal{S}$, action space $\mathcal{A}$, reward function $\mathcal{R}$, transition dynamics $\mathcal{P}$, and initial state distribution μ .

Sensor Library and Resource Costs The agent can observe the environment through a library of sensors $\Phi = \{\varphi_1, \dots, \varphi_n\}$. Here, a “sensor” refers broadly to any processed physical measurement, including operations like filtering or feature extraction. Mathematically, each sensor φ_j yields an observation in $\mathcal{O}_j$ distributed according to a history-dependent map $\varphi_j : \mathcal{S}^* \times \mathcal{A}^* \rightarrow \Delta(\mathcal{O}_j)$. Activating a subset of sensors $\phi \subseteq \Phi$ incurs a per-timestep resource cost $C(\phi) \in \mathbb{R}^+$ that represents the associated physical overheads such as energy, computation, and communication.

Programmatic Specification of Mode-Switching Controllers (MSC) To enable designers to systematically exploit the non-stationary resource requirements of a task, we adopt a flexible programmatic specification of MSC designs structured as a Finite State Machine (FSM) $\mathcal{D} := (\Sigma, \Gamma, \delta, \nu)$:

- **Control Modes** (Σ): A discrete set of operating control modes, $\Sigma = \{\sigma^1, \dots, \sigma^N\}$
- **Resource Allocation** (Γ): Maps control modes to their active sensory-compute configuration, $\Gamma : \Sigma \rightarrow 2^\Phi$
- **Transition Logic** (δ): Mode transition functions based on an interpretable environment state, $\delta : \Sigma \times \mathcal{S} \rightarrow \Sigma$
- **Initial Mode** (ν): Determines initial mode, $\nu : \mathcal{S} \rightarrow \Sigma$

```
class Design:
    def mode_obs(self) -> dict[int, Config]:
        return {
            0: Config(...),
            1: Config(...),
        }

    def mode_transitions(self)
    -> list[tuple[int, int, callable[[], bool]]]:
        def transition_0_1(state) -> bool: ...
        return [(0, 1, transition_0_1)]

    def initial_mode(self, state) -> int:
        return 0
```

Design Objective The designer’s goal is to synthesize a design $\mathcal{D}$ that minimizes the resource cost $R(\mathcal{D})$, while ensuring the resulting MSC satisfies a minimum performance requirement $J(\mathcal{D}) \geq \epsilon$. Let $\tau = (s_0, \sigma_0, a_0, r_0, \dots, s_H, \sigma_H, a_H, r_H)$ denote a trajectory of states, modes, actions, and rewards generated by executing the MSC in MDP $\mathcal{M}$ for H steps. The design objective is:

$$\arg \min_{\mathcal{D} \in \mathbb{D}} R(\mathcal{D}) = \mathbb{E}_\tau \left[\sum_{t=0}^H C(\Gamma(\sigma_t)) \right] \quad \text{s.t.} \quad J(\mathcal{D}) = \mathbb{E}_\tau \left[\sum_{t=0}^H r_t \right] \geq \epsilon$$

Reinforcement Learning Of Mode-Switching Controllers To obtain the resource-performance characteristics of a design $\mathcal{D}$, the designer has to run a synthesis procedure to instantiate an MSC for evaluation. The MSC $\Pi_{\mathcal{D}}$ has to operate on local mode-specific observations $\mathcal{O}_\sigma$, determined by $\Gamma(\sigma)$, to produce control actions and mode-switching decisions. Mathematically, $\Pi_{\mathcal{D}}$ is a collection of mode-specific controllers π and monitors λ , $\Pi_{\mathcal{D}} = \{(\pi_\sigma, \lambda_\sigma) : \mathcal{O}_\sigma \rightarrow \mathcal{A} \times \Sigma \mid \sigma \in \Sigma\}$. The synthesis procedure should attempt to generate a $\Pi_{\mathcal{D}}$ that maximizes the task performance J , while adhering to the sensory-compute constraints specified by the design. Given the versatility of end-to-end RL, we propose MSC-RL as a generic algorithm to synthesize MSCs for tasks with access to simulators.

MSC-RL adopts an asymmetric actor-critic paradigm [12] and instantiates a privileged critic network and a mixture-of-experts style architecture to represent the MSC $\Pi_{\mathcal{D}}$ – where each mode-specific controller π_σ and monitor λ_σ is represented by separate networks. MSC-RL involves three stages of training: **[Stage I]** Under the privileged mode-transition function δ specified by the design $\mathcal{D}$, run any RL algorithm, PPO [13] in our case, to optimize the collection of controllers $\{\pi_\sigma\}$ to maximize $J(\mathcal{D})$. **[Stage II]** Run the best-performing checkpoint found in Stage I with privileged mode-transition function δ to curate a dataset containing triples: current mode, observation, and next mode $\{(\sigma_t, o_t, \sigma_{t+1})_{t=0}^H\}$. Train the monitors λ_σ to mimic the mode transitions with some temporal label smoothing [14], as illustrated in Figure 3. **[Stage III]** Keeping the monitors λ frozen, fine-tune the controllers through RL to adapt to the mode transitions flagged by the learned monitors [15]. Across Stages I and III, the MSC controller is frequently evaluated and checkpointed – the best-performing checkpoint is returned as the representative MSC of the design $\mathcal{D}$. As feedback, the designer gets

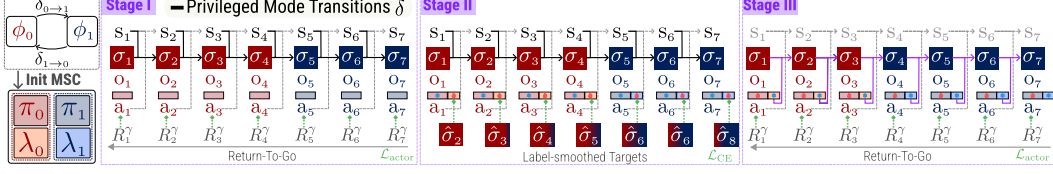


Figure 3: Computation graph of the three stages in MSC-PPO. [Stage I] Mode-transitions unroll according to privileged δ specified by the design $\mathcal{D}$, corresponding policy networks π_{σ_t} are activated to sample actions and receive policy gradients. [Stage II] Traces from the learned policy are used to train mode-specific monitors λ_{σ_t} to mimic δ with temporal label smoothing [Stage III] Fine-tunes the policy π to adapt to the mode-transitions determined by λ

109 auxiliary performance metrics such as expected resource cost $R(\mathcal{D})$, mode occupancy (% time spent
110 in each mode), etc., by evaluating this representative MSC.

111 The controller synthesis procedure is an important part of the design loop, as the assertions on resource-
112 performance tradeoffs are limited by the expressivity and learnability of the chosen controller class.
113 Additionally, we make choices such as (a) shared neural architecture for controllers/monitors, (b)
114 zero-padding observations to match maximum observation size, for faster training with JAX JIT [16]
115 compilation. More implementation details are provided in Appendix A.1

116 **The LLM-Guided Search Procedure** The design space, including modes, sensor configurations,
117 and mode transition functions, is combinatorially large. Brute force search is infeasible, and so are
118 gradient-based approaches owing to the various discrete choices involved. However, recent advances
119 in LLM-in-loop algorithmic optimization [17, 18] offer a viable path to tackle this challenge. These
120 works demonstrate that the semantic priors and reasoning capabilities of modern coding LLMs can
121 be harnessed to serve as effective operators to hypothesize and refine interpretable programs [19], as
122 evidenced in demonstrations of optimizing algorithmic heuristics [17, 18], reward functions [20], and
123 curricula [21]. We build on these insights to propose LEMCA, LLM-Guided synthesis of Efficient
124 Mode-Switching Control Architectures, an LLM-guided evolutionary search pipeline to synthesize
125 and refine MSC designs represented as Python programs. The LLM’s capabilities are leveraged in
126 two ways as part of the search process: (a) proposing structured edits to synthesize new designs that
127 are likely to improve the objective or gather new insights, and (b) summarizing the insights from
128 evaluations into a compact Design Log, tracking observed trade-offs and open directions, to guide
129 the refinement process – the prompts for these operations are outlined in Appendix A.2. The design
130 loop utilizes a genetic algorithm (GA) with a feasible-first [22] tournament selection [23, 24]; with
131 the crossover handled by an LLM. Due to space limitations, we present the details in Appendix A.3.

132 3 Experiments

133 We evaluate LEMCA across the following tasks with diverse sensory-compute parameterizations:

134 **point-to-disk:** We revisit the task introduced in Sec. 1, where a velocity-controlled robot must
135 be steered into a target disk of radius $r^* = 0.01$ as quickly as possible while balancing the costs
136 associated with sensing. The designer can configure sensors that reveal various slices of the state,
137 such as (a) Cartesian position, (b) Radial distance, and (c) Sector presence. Each sensor type offers
138 distinct tunable noise levels $(\sigma_{xy}, \sigma_r, \sigma_\theta)$ and discretizations. For instance, the designer can specify the
139 number of sectors n_{sec} to determine the angular resolution, or define a sequence of radial boundaries
140 $(d_1, d_2, \dots, d_n)$ to dictate radial distance discretizations. These configurations naturally introduce
141 different cost-benefit tradeoffs – for instance, a high-resolution Sector detector can help navigate
142 the robot straight into the target disk (minimizing detours), but requires expensive, high-resolution
143 radar scans. To formalize this, we adopt an information-theoretic cost as a proxy for the per-step
144 resource consumption for each sensor. We implement a linear-additive cost model that penalizes
145 the information revealed by a sensor configuration φ , as $C(\varphi) = \alpha I(S; \varphi) + \beta$ (see Appendix B
146 for more details). To demonstrate that LEMCA can adapt to varied cost structures, we evaluate it

under two cost specifications: `CostlyGPS`, where `RadialSector` sensors are substantially cheaper than `Cartesian` sensors, and `CheapGPS`, where certain `Cartesian` configurations are priced competitively. These setups mirror real-world constraints where sensor modalities carry highly asymmetric costs, often dictated by environmental factors like GPS availability or hardware complexity.

RF-DMC: We consider rangefinder-variants of tasks `carpole-swingup`, `cup-catch`, and `finger-spin` from the standard `dm_control` [11, 25] suite. Here instead of the state the agent has to operate on 360° rangefinder scans from a fixed site – tip of the pole, center of the cup, and finger-tip. These variants capture many practical characteristics of real systems such as noisy perception, non-linear dynamics, non-linear sensor fusion, etc., while offering a fast-to-simulate setup to study the impacts of sensor noise [26], resolution, and history. The designer is allowed to tune the quality q , resolution n , and history buffer size h of rangefinder scan for each mode. The quality refers to discrete operating points that map to distinct noise characteristics, with **A** being the highest grade (minimal noise) and **F** the lowest, high-noise configuration. Each operating point, with associated SNR, determines the energy consumption of a single rangefinder measurement. The per-step resource cost of a configuration scales as: $n \cdot (\text{Energy}(q) + c_{\text{mem}} \log h)$, see Appendix C. Task performance is measured by the episodic return normalized by the oracle’s episodic return. The returns track the fraction of the episode where the pole is balanced upright, the ball is caught, or the spinner spins above a target velocity – each condition offers some slack without qualitatively altering the behavior.

clutter-nav: We consider a 2D planar navigation task in which a disk-shaped robot, governed by unicycle dynamics and equipped with a noisy radar sensor, must navigate to randomly sampled goals while avoiding other static and dynamic disk-shaped obstacles. The onboard radar sensor detects obstacles within its line of sight and reports their positions and approach velocities subject to measurement noise. Control synthesis is driven by a Model Predictive Path Integral (MPPI) [27] planner operating under a certainty-equivalence assumption, which treats the noisy radar scans as the true state of the environment. Additionally, the mode-transition functions or monitors specified in the design operate directly over these noisy radar scans – alleviating the need for any learning. The design space allows the joint configuration of both Sensor and Planner parameters. For the Sensor, the configurable parameters include: reliable detection range R , the maximum range r_{max} , and the field of view $\vartheta \in [\pi/3, 2\pi]$. The Planner configuration dictates the number of sampled rollouts N , horizon H , refinement iterations I , and re-plan period p (where the planner is invoked every p steps to amortize sensing and compute overheads). Consequently, the per-step resource cost is modeled as a weighted sum of the energy required for reliable sensing ($\propto R^2$), the sensor coverage area ($\propto r_{\text{max}}^2 \cdot \vartheta$), and the computational load ($\propto N \cdot H \cdot I/p$). Finally, the task performance is quantified as the *reciprocal* of the 90th percentile collision rate across 1000 evaluation episodes, subject to a minimum success rate of 80%; see Appendix D for more details.

LEMCA discovers cost-effective MSCs Across tasks, we find that LEMCA consistently identifies MSC designs that push the frontier spanned by monolithic designs obtained by a dense parameter sweep (Figure 4). In most settings, LEMCA discovers MSCs that reduce the resource costs by $\geq 20\%$ compared to suitable cost-effective monolithic designs. Notably, in the `clutter-nav` task, we observe 10x reductions in resource costs, highlighting the potential of MSC controllers to exploit the non-stationary resource requirements of tasks and the effectiveness of LEMCA in synthesizing such strategies. For the RF-DMC tasks, we additionally visualize the trajectory of designs synthesized by LEMCA on the performance-resource plane in Figure 4 (bottom row). This illustrates that LEMCA consistently expands the Pareto frontier of MSC designs over generations in the direction that pushes for resource efficiency subject to performance constraint. For `point-to-disk[CostlyGPS]`, we additionally contrast the best designs found by LEMCA with those found by running BayesOpt [28] on parameterized templates that incorporate structures found in LLM-synthesized designs. While the best designs found by BayesOpt (BO-MS) show marginal gains in resource costs, arriving at those designs takes more evaluations (≈ 100) and requires pre-specified templates (see Appendix E.1).

LEMCA discovers cost-adaptive designs Across the evaluated tasks, LEMCA consistently synthesizes highly performant and effective MSC designs by exploiting task and cost structures. For

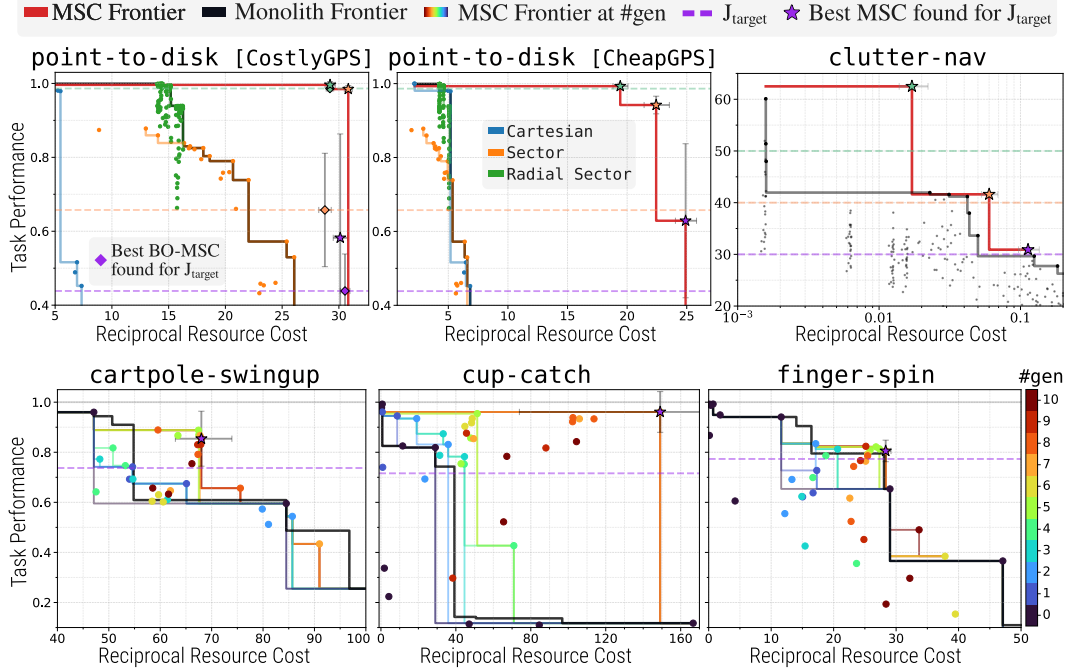


Figure 4: Performance-Resource curve for the evaluated tasks. LEMCA consistently identifies MSC designs that dominate the Monolith frontier. The errorbars depict $\pm 1\sigma$ bands of 1000 rollouts.

198 point-to-disk, it recognizes that sensing is entirely inconsequential once the robot enters the
 199 target disk. Under the CostlyGPS cost structure, LEMCA adopts a two-mode strategy using
 200 RadialSector sensors (Figure 5-(a,b)) for $J_{\text{target}} = 0.44$ and 0.98 – partitioning the state space
 201 into a target disk mode and an outer mode. Based on the performance target, LEMCA determines
 202 suitable Sector and Radial discretizations to orchestrate a progressive braking strategy. Conversely,
 203 for point-to-disk[CheapGPS] at $J_{\text{target}} = 0.66$, the best design (Figure 5-(c)) leverages affordable
 204 Cartesian sensors along with Radial bands to orchestrate a “foveation” strategy, deploying more
 205 expensive, less noisy Cartesian sensors closer to the target. In cartpole-swingup, LEMCA settles
 206 on a dual-mode strategy (Figure 5-(d)) matching the natural “swingup” and “balance” phases,
 207 utilizing a higher-quality rayscan to swing the pole up before transitioning to a noisier rayscan
 208 for upright stabilization. For cup-catch (Figure 5-(e)), a three-mode strategy is dictated by the
 209 L_∞ distance between the ball and the cup (tethered by a 0.3m rope); the MSC activates only four
 210 low-quality rangefinders when the rope is nearly taut and dynamics are highly predictable, transitions
 211 to a higher-quality, higher-resolution rayscan as the ball swings over the cup for the catch, and shuts
 212 off the sensors entirely once successful. In finger-spin (Figure 5-(f)), the system implements
 213 a two-mode strategy that starts with the cost-optimal monolith configuration to strike the spinner
 214 above the target velocity of 15 rad/s and switches to a cheaper, lower-quality configuration between
 215 subsequent strikes. Finally, for clutter-nav (Figure 5-(g)), a three-mode configuration exploits
 216 the natural task affordances: a “cruise” mode uses cheaper Planner settings and slower re-planning
 217 when no obstacles are in sight, an “alert” mode switches to active steering when an obstacle nears,
 218 and a high-compute “panic” mode maximizes reactivity if the agent becomes dangerously close to an
 219 obstacle. These just highlight a few salient designs; more details and rollout visualizations can be
 220 found on the project website: <https://lemca-robotics.github.io/>

221 **LEMCA performs effective refinement** In Figure 6, we visualize the sequential design decisions
 222 made by LEMCA throughout the refinement process for the cartpole-swingup task. LEMCA be-
 223 gins by establishing reasonable initial choices for both the mode configurations and boundaries. It sub-
 224 sequently experiments with adjustments to various configuration parameters—namely quality, history,
 225 and mode boundaries—ultimately arriving at the final configuration illustrated in Figure 5(d). This

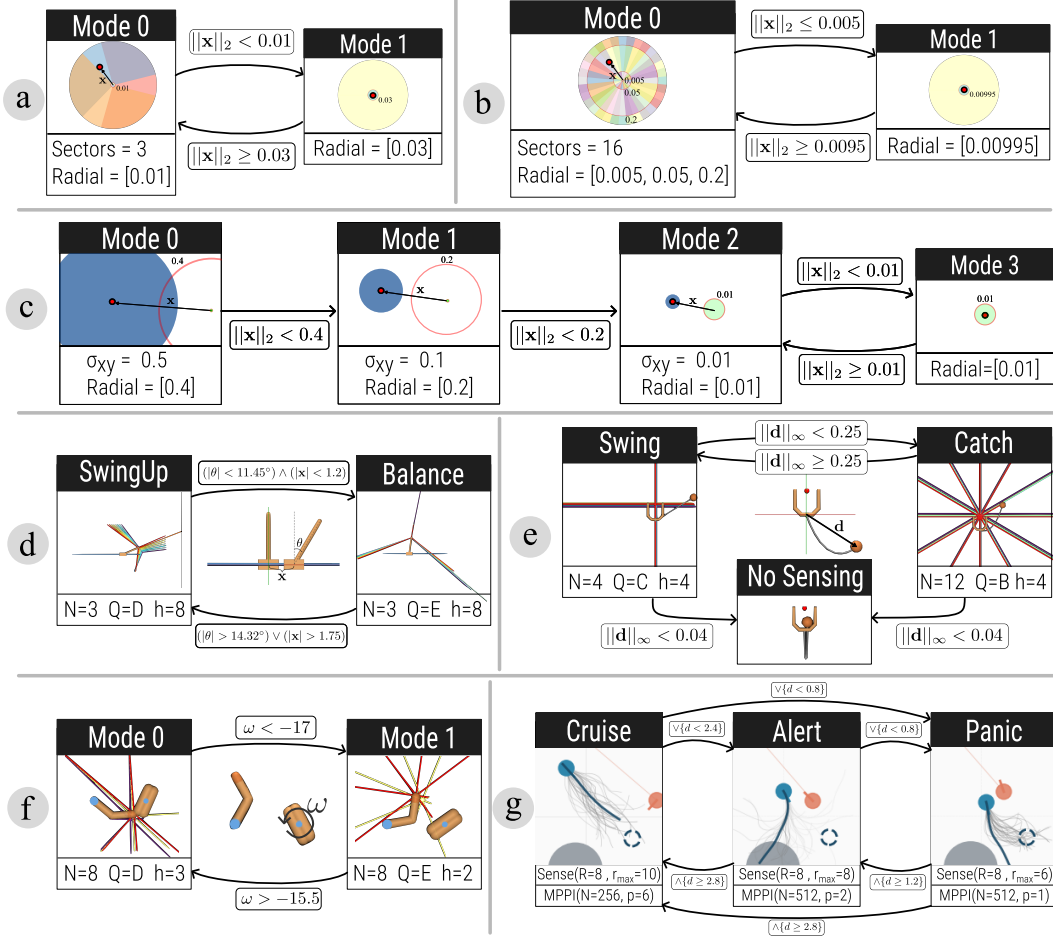


Figure 5: Salient MSC designs identified by LEMCA for different tasks. More designs and rollout videos are available at <https://lemca-robotics.github.io/>.

226 execution trace demonstrates that modern reasoning and code-generation LLMs (gemini-3.1-pro
 227 in our experiments) can effectively navigate the design space to optimize for resource efficiency.
 228 Although we implement a GA + Design Log framework for LEMCA, many other agentic loops
 229 merit future exploration. Our ablation study of alternative LLM-in-the-loop variants (Appendix E.2)
 230 demonstrates that the GA framework is essential, while the influence of the design log is inconclusive.

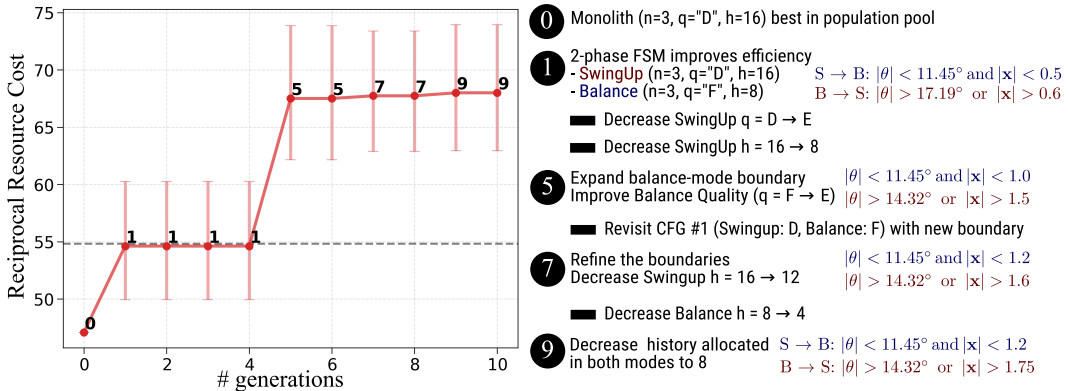


Figure 6: Design decisions and refinements by LEMCA over generations for cartpole-swingup.

4 Related Work

Resource-Efficient Hybrid, Event- and Self-Triggered Control. Hybrid control and event-triggered control provide a framework to optimize resource consumption of controllers by exploiting task-specific dynamics. For example, hybrid impulse systems restrict actuation to critical instants to conserve propellant [29], alternate between planning and reactive policies for navigation [8], or switch sensory modalities based on contact state [30]. Similarly, networked control methods—including Lebesgue sampling [31] and event- or self-triggered scheduling [32, 33, 7]—reduce communication bandwidth by restricting feedback to threshold-crossing events. A fundamental limitation across these frameworks is that the event specifications, available modes, and resource strategies are hand-crafted by experts. Recent works employing learning within these frameworks [6, 34, 35, 36] typically optimize isolated parameters like trigger thresholds or sampling rates. We propose a more holistic, automated pipeline to jointly synthesize control modes, their transitions, and resource allocations that are tailored to the task performance-cost specifications.

Hierarchical RL. Task decomposition [37, 38] and behavior chaining [10, 39, 15, 40] traditionally target long-horizon policy learning and skill re-usability. However, these hierarchical frameworks typically ignore the operational overhead of the behaviors themselves, and default to monolithic configurations across skills. In contrast, we use the MSC specification to iteratively optimize a controller’s inference-time sensory-compute resources.

LLMs for Program Synthesis. A growing literature uses code-generating LLMs as operators inside evolutionary search. Eureka [20] evolves reward code to exceed expert reward engineering. FunSearch [41], AlphaEvolve [17], and ShinkaEvolve [18] generalize this to the discovery of algorithmic artifacts. LEMCA inherits the LLM-as-evolutionary-operator methodology from this lineage but differs in its target: LEMCA synthesizes MSC programs under a constrained optimization setting, aiming to minimize resource consumption while retaining functionality.

5 Discussion

By framing the design of MSCs as a program synthesis problem, LEMCA demonstrates how LLM-guided evolutionary search can automate the synthesis of efficient-yet-performant MSCs across tasks with different dynamics, sensory modalities, cost structures, and synthesis procedures.

Limitations This first iteration of our framework makes a few simplifying assumptions: (a) the monitor and controllers in a mode share the same sensor suite, (b) the cost of synthesizing actions and switching decisions is homogeneous, and (c) mode-switching costs are negligible. While these assumptions limit the scope of designs that can be explored, it is straightforward to extend LEMCA to decouple configurations and factor the non-homogeneous cost characteristics. Next, our resource-performance characterizations are fundamentally bounded by the synthesis procedure. The expressivity and learnability of the synthesis procedure, such as MSC-PP0, determine what tradeoffs are realizable – an unreliable synthesis can mislead the search. Additionally, with MSC-PP0 each evaluation requires a full training run, making the pipeline computationally expensive when simulation is costly – RF-DMC tasks require ≈ 0.5 L40 GPU hours per evaluation. Finally, our ablations of the designer loop suggest that while the design log generally aids refinements, it can, in some settings, limit exploration and result in suboptimal choices – suggesting more principled approaches to context management might be needed depending on the problem.

Future Directions While the proposed pipeline primarily focuses on resource allocation as a discrete optimization problem handled by an LLM, a promising avenue for future work is integrating continuous optimization tools – such as BayesOpt or gradient-based optimization for templated design parameterizations – to alleviate the LLM’s computational burden. Additionally, incorporating more efficient and reliable controller synthesis algorithms [42, 43] represents an important step in scaling this pipeline to a broader range of tasks.

References

- [1] M. Erdmann. Understanding action and sensing by designing action-based sensors. *The International Journal of Robotics Research*, 14(5):483–509, 1995.
- [2] B. R. Donald. On information invariants in robotics. *Artificial Intelligence*, 72(1-2):217–304, 1995.
- [3] S. M. LaValle. Sensing and filtering: A fresh perspective based on preimages and information spaces. *Foundations and Trends® in Robotics*, 1(4):253–372, 2012.
- [4] G. McFassel and D. A. Shell. Every action-based sensor. In *International Workshop on the Algorithmic Foundations of Robotics*, pages 176–193. Springer, 2020.
- [5] A. Majumdar, Z. Mei, and V. Pacelli. Fundamental limits for sensor-based robot control. *The International Journal of Robotics Research*, 42(12):1051–1069, 2023.
- [6] A. Krishna, E. S. Hu, and D. Jayaraman. The value of sensory information to a robot. In *The Thirteenth International Conference on Learning Representations*, 2025.
- [7] W. P. Heemels, K. H. Johansson, and P. Tabuada. An introduction to event-triggered and self-triggered control. In *2012 IEEE 51st IEEE conference on decision and control (cdc)*, pages 3270–3285. IEEE, 2012.
- [8] D. Nakhaeinia, P. Payeur, T. S. Hong, and B. Karasfi. A hybrid control architecture for autonomous mobile robot navigation in unknown dynamic environment. In *2015 IEEE International Conference on Automation Science and Engineering (CASE)*, pages 1274–1281. IEEE, 2015.
- [9] R. Fierro, A. Das, V. Kumar, and J. Ostrowski. Hybrid control of formations of robots. In *Proceedings 2001 ICRA. IEEE International Conference on Robotics and Automation (Cat. No.01CH37164)*, volume 1, pages 157–162 vol.1, 2001. doi:10.1109/ROBOT.2001.932546.
- [10] R. R. Burridge, A. A. Rizzi, and D. E. Koditschek. Sequential composition of dynamically dexterous robot behaviors. *The International Journal of Robotics Research*, 18(6):534–555, 1999.
- [11] S. Tunyasuvunakool, A. Muldal, Y. Doron, S. Liu, S. Bohez, J. Merel, T. Erez, T. Lillicrap, N. Heess, and Y. Tassa. dm_control: Software and tasks for continuous control. *Software Impacts*, 6:100022, 2020. ISSN 2665-9638. doi:<https://doi.org/10.1016/j.simpa.2020.100022>. URL <https://www.sciencedirect.com/science/article/pii/S2665963820300099>.
- [12] L. Pinto, M. Andrychowicz, P. Welinder, W. Zaremba, and P. Abbeel. Asymmetric actor critic for image-based robot learning. *arXiv preprint arXiv:1710.06542*, 2017.
- [13] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [14] R. Müller, S. Kornblith, and G. E. Hinton. When does label smoothing help? *Advances in neural information processing systems*, 32, 2019.
- [15] Y. Chen, C. Wang, L. Fei-Fei, and K. Liu. Sequential dexterity: Chaining dexterous policies for long-horizon manipulation. In *7th Annual Conference on Robot Learning*, 2023.
- [16] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, Y. Katariya, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang. JAX: composable transformations of Python+NumPy programs, 2018. URL <http://github.com/jax-ml/jax>.

- [17] A. Novikov, N. Vű, M. Eisenberger, E. Dupont, P.-S. Huang, A. Z. Wagner, S. Shirobokov, B. Kozlovskii, F. J. Ruiz, A. Mehrabian, et al. Alphaevolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.
- [18] R. T. Lange, Y. Imajuku, and E. Cetin. Shinkaevolve: Towards open-ended and sample-efficient program evolution. *arXiv preprint arXiv:2509.19349*, 2025.
- [19] Y. Zhang, S. A. Khan, A. Mahmud, H. Yang, A. Lavin, M. Levin, J. Frey, J. Dunnmon, J. Evans, A. Bundy, et al. Exploring the role of large language models in the scientific method: from hypothesis to discovery. *npj Artificial Intelligence*, 1(1):14, 2025.
- [20] Y. J. Ma, W. Liang, G. Wang, D.-A. Huang, O. Bastani, D. Jayaraman, Y. Zhu, L. Fan, and A. Anandkumar. Eureka: Human-level reward design via coding large language models. *arXiv preprint arXiv:2310.12931*, 2023.
- [21] J. Ma, W. Liang, H.-J. Wang, Y. Zhu, L. Fan, O. Bastani, and D. Jayaraman. Dreureka: Language model guided sim-to-real transfer. *RSS*, 2024.
- [22] K. Deb. An efficient constraint handling method for genetic algorithms. *Computer methods in applied mechanics and engineering*, 186(2-4):311–338, 2000.
- [23] D. E. Goldberg and K. Deb. A comparative analysis of selection schemes used in genetic algorithms. In *Foundations of genetic algorithms*, volume 1, pages 69–93. Elsevier, 1991.
- [24] B. L. Miller, D. E. Goldberg, et al. Genetic algorithms, tournament selection, and the effects of noise. *Complex systems*, 9(3):193–212, 1995.
- [25] K. Zakka, B. Tabanpour, Q. Liao, M. Haiderbhai, S. Holt, J. Y. Luo, A. Allshire, E. Frey, K. Sreenath, L. A. Kahrs, C. Sferrazza, Y. Tassa, and P. Abbeel. Mujoco playground: An open-source framework for gpu-accelerated robot learning and sim-to-real transfer., 2025. URL https://github.com/google-deepmind/mujoco_playground.
- [26] S. Thrun, W. Burgard, and D. Fox. *Probabilistic robotics*. Intelligent robotics and autonomous agents. MIT Press, 2005. ISBN 978-0-262-20162-9.
- [27] G. Williams, A. Aldrich, and E. Theodorou. Model predictive path integral control using covariance variable importance sampling. *arXiv preprint arXiv:1509.01149*, 2015.
- [28] M. Olson, E. Santorella, L. C. Tiao, S. Cakmak, M. Garrard, S. Daulton, Z. J. Lin, S. Ament, B. Beckerman, E. Onofrey, et al. Ax: A platform for adaptive experimentation. In *AutoML 2025 ABCD Track*, 2025.
- [29] D. R. Wibben and R. Furfaro. Terminal guidance for lunar landing and retargeting using a hybrid control strategy. *Journal of Guidance, Control, and Dynamics*, 39(5):1168–1172, 2016.
- [30] Z. Zhao, S. Haldar, J. Cui, L. Pinto, and R. Bhirangi. Touch begins where vision ends: Generalizable policies for contact-rich manipulation. *arXiv preprint arXiv:2506.13762*, 2025.
- [31] K. J. Astrom and B. M. Bernhardsson. Comparison of riemann and lebesgue sampling for first order stochastic systems. In *Proceedings of the 41st IEEE Conference on Decision and Control, 2002.*, volume 2, pages 2011–2016. IEEE, 2002.
- [32] P. Tabuada. Event-triggered real-time scheduling of stabilizing control tasks. *IEEE Transactions on Automatic control*, 52(9):1680–1685, 2007.
- [33] A. Anta and P. Tabuada. To sample or not to sample: Self-triggered control for nonlinear systems. *IEEE Transactions on automatic control*, 55(9):2030–2042, 2010.
- [34] D. Baumann, J.-J. Zhu, G. Martius, and S. Trimpe. Deep reinforcement learning for event-triggered control. In *2018 IEEE Conference on Decision and Control (CDC)*, pages 943–950. IEEE, 2018.

- [35] H. Sun et al. Deep reinforcement learning of event-triggered communication and consensus-based control for distributed cooperative transport. *Robotics and Autonomous Systems*, 159: 104307, 2022.
- [36] L. Treven, B. Sukhija, Y. As, F. Dörfler, and A. Krause. When to sense and control? a time-adaptive approach for continuous-time rl. *Advances in Neural Information Processing Systems*, 37:63654–63685, 2024.
- [37] R. S. Sutton, D. Precup, and S. Singh. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112(1-2):181–211, 1999.
- [38] M. Stolle and D. Precup. Learning options in reinforcement learning. In *International Symposium on abstraction, reformulation, and approximation*, pages 212–223. Springer, 2002.
- [39] A. Bagaria and G. Konidaris. Option discovery using deep skill chaining. In *International Conference on Learning Representations*, 2019.
- [40] L. Marzari, A. Pore, D. Dall’Alba, G. Aragon-Camarasa, A. Farinelli, and P. Fiorini. Towards hierarchical task decomposition using deep reinforcement learning for pick and place subtasks. In *2021 20th International Conference on Advanced Robotics (ICAR)*, pages 640–645. IEEE, 2021.
- [41] A. Fawzi and B. R. Paredes. Funsearch: Making new discoveries in mathematical sciences using large language models. *DeepMind Google*, 2023.
- [42] E. S. Hu, J. Springer, O. Rybkin, and D. Jayaraman. Privileged sensing scaffolds reinforcement learning. *arXiv preprint arXiv:2405.14853*, 2024.
- [43] M. Hussing, L. G. d’Aliberti, C. Voelcker, B. Eysenbach, and E. Eaton. Behavior-consistent deep reinforcement learning. *arXiv preprint arXiv:2605.21214*, 2026.
- [44] DeepMind, I. Babuschkin, K. Baumli, A. Bell, S. Bhupatiraju, J. Bruce, P. Buchlovsky, D. Budden, T. Cai, A. Clark, I. Danihelka, A. Dedieu, C. Fantacci, J. Godwin, C. Jones, R. Hemsley, T. Hennigan, M. Hessel, S. Hou, S. Kapturowski, T. Keck, I. Kemaev, M. King, M. Kunesch, L. Martens, H. Merzic, V. Mikulik, T. Norman, G. Papamakarios, J. Quan, R. Ring, F. Ruiz, A. Sanchez, L. Sartran, R. Schneider, E. Sezener, S. Spencer, S. Srinivasan, M. Stanojević, W. Stokowiec, L. Wang, G. Zhou, and F. Viola. The DeepMind JAX Ecosystem, 2020. URL <http://github.com/google-deepmind>.
- [45] P. Kidger and C. Garcia. Equinox: neural networks in JAX via callable PyTrees and filtered transformations. *Differentiable Programming workshop at Neural Information Processing Systems 2021*, 2021.
- [46] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.
- [47] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel. High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*, 2015.
- [48] K. W. Cobbe, J. Hilton, O. Klimov, and J. Schulman. Phasic policy gradient. In *International Conference on Machine Learning*, pages 2020–2027. PMLR, 2021.
- [49] mozilla ai. any-llm: Communicate with an llm provider using a single interface, 2026. URL <https://github.com/mozilla-ai/any-llm>.
- [50] A. Censi. A mathematical theory of co-design. *arXiv preprint arXiv:1512.08055*, 2015.
- [51] M.-P. Neumann, G. Zardini, A. Cerofolini, and C. H. Onder. On the co-design of components and racing strategies in formula 1. In *2024 IEEE Intelligent Vehicles Symposium (IV)*, pages 2876–2881, 2024. doi:10.1109/IV55156.2024.10588533.

- 408 [52] J. M. O’Kane and S. M. LaValle. On comparing the power of robots. *The International Journal*
409 *of Robotics Research*, 27(1):5–23, 2008.
- 410 [53] M. Egerstedt and R. W. Brockett. Motion description languages and hybrid architectures for
411 motor control. In *Proceedings of the 38th IEEE Conference on Decision and Control*, 1999.

Table of Contents

Appendix A: Implementation Details

A.1: MSC-PPO: Overview and Hyperparameters (page 13)

A.2: Prompts used in LEMCA (page 15)

A.3: LEMCA: Overview and Hyperparameters (page 20)

Task Details

Appendix B: point-to-disk (page 22)

Appendix C: RF-DMC (rangefinder-variants of dm_control tasks) (page 24)

Appendix D: clutter-nav (page 26)

Appendix E: Points of comparison

E.1: B0-MSC: BayesOpt on point-to-disk[CostlyGPS] (page 29)

E.2: Alternative LLM-in-the-loop mechanisms (page 30)

Appendix F: Extended Related Work

Project Website

Link: <https://lemca-robotics.github.io/>

The website contains interactive media of LEMCA generated designs and its rollouts.

A Implementation Details

A.1 MSC-PPO

We use an end-to-end JAX-JIT compiled PPO [13] implementation as the underlying RL trainer-loop for synthesizing MSCs in tasks point-to-disk and RF-DMC. The implementation is based on open source libraries: JAX [16], Optax [44], Equinox [45], and mujoco-playground [25]. The high-level overview of the MSC-PPO trainer is described in Algorithms 1 and 2 and the hyperparameters are outlined in Table 1. The code will be made available on the website upon publication.

Architecture MSC-PPO implements an asymmetric actor-critic paradigm [12]. The MSC $\Pi_{\mathcal{D}}$ is represented as a collection of actor π_{σ} and monitor λ_{σ} networks, each sharing the same architecture to facilitate JIT compilation. Similarly, the mode-specific observations are zero-padded to a common (maximum) dimension to fix shapes for JIT compilation, and a fixed-length FIFO buffer supplies sensory readings for the requested mode-specific h history length. A single *privileged* critic V operates on the full environment state s_t and is shared across modes and represents two-hot encoded value targets over a fixed symlog support [46]. The actors ingest mode-specific observations to represent a tanh-squashed action distribution with state-dependent variance represented as a diagonal Gaussian – all tasks have a bounded action space $[-1, 1]^{|A|}$. The monitors operate on mode-specific observations and executed action to produce logits over the next N modes. The networks are MLPs that use LayerNorm post-activations – the depth and width are configured per task (Table 1). The networks operate on normalized observations with per-mode normalization statistics tracked online over rollouts.

Training The three stages of MSC-PPO (Sec. 2) share a common rollout-and-update structure (Figure 3, Algorithm 1). Rollouts collect S steps across E parallel environments and compute advantages with GAE [47]. We adopt a phasic, decoupled update [48]: where for every batch ($E \times S$) the critic is updated for N_V epochs and the (active) actors for N_{π} epochs, each epoch involves running gradient descent over randomly permuted and grouped minibatches of size M .

[Stage I] trains $\{\pi_{\sigma}\}, V$ under the privileged transition δ . [Stage II] freezes the policy, transitions modes according to δ , and trains monitors $\{\lambda_{\sigma}\}$ to predict the next mode by minimizing cross-

454 entropy against temporally label-smoothed targets [14] for N_λ epochs. The smoothing (Algorithm 2)
 455 convolves the one-hot next-mode sequence with a 1-D Gaussian kernel of width 7 along time, and
 456 renormalizes the categorical distribution over modes for each step. This blends mode labels within
 457 a ± 3 -step window around each transition so that the monitor can learn softer decision boundaries
 458 near impending mode switches. [Stage III] freezes the monitors and fine-tunes $\{\pi_\sigma\}$, V with the
 459 learned transitions $\sigma_{t+1} = \arg \max_\sigma \lambda_{\sigma_t}(o_t, a_t)$.

460 Networks are optimized with AdamW (Nesterov Momentum), with the weight decay applied only
 461 to the weight matrices. The biases, normalization scales, and policy log-std use vanilla Adam.
 462 The optimizer is configured to use global gradient-norm clipping with learning-rate warmup sched-
 463 ules (Table 1) that are restarted every stage. Throughout Stages I and III the controller is periodically
 464 evaluated and the checkpoint with the best episodic return is carried forward to the next stage; the
 465 final best checkpoint is returned as the representative MSC $\Pi_{\mathcal{D}}$ for the training run. When running
 466 MSC-PPO with multiple seeds, we pick the best performing checkpoint (based on episodic return)
 467 across seeds as the representative MSC for that design.

Algorithm 1 MSC-PPO: Controller synthesis for a design $\mathcal{D}$

Input: MDP $\mathcal{M}$, design $\mathcal{D} = (\Sigma, \Gamma, \delta, \nu)$, sensor library Φ

Networks:

per-mode actors $\{\pi_\sigma : \mathcal{O}_\sigma \rightarrow \Delta(\mathcal{A})\}$, per-mode monitors $\{\lambda_\sigma : \mathcal{O}_\sigma \rightarrow \Delta(\Sigma)\}$, privileged critic $V : \mathcal{S} \rightarrow \mathbb{R}$

Parameters: #envs E , rollout length S , #batches N_I, N_{II}, N_{III} , #epochs N_V, N_π, N_λ

```

1: procedure ROLLOUT(transition_fn) ▷  $E \times S$  transitions
2:    $\sigma_0 \leftarrow \nu(s_0)$ ; at each  $t$ :  $o_t \sim \Gamma(\sigma_t)$ ,  $a_t \sim \pi_{\sigma_t}(\cdot | o_t)$ , env.step  $\mathcal{M}$ ,  $V$  on privileged  $s_t$ 
3:    $\sigma_{t+1} \leftarrow \text{transition\_fn}(\sigma_t, o_t, a_t, s_{t+1})$ 
4:   return transitions  $\{(s_t, r_t, \sigma_t, o_t, a_t)\}$ 
5: end procedure

6: // [Stage I]: train actors under privileged transitions
7: for  $k = 1$  to  $N_I$  do
8:    $\mathcal{B} \leftarrow \text{ROLLOUT}((\sigma, \_, \_, s') \mapsto \delta(\sigma, s'))$ ; compute GAE
9:   update  $V$  for  $N_V$  epochs, then update active actors  $\{\pi_{\sigma_t}\}$  for  $N_\pi$  epochs (PPO)
10:  periodically evaluate; track best-return checkpoint
11: end for; [load best checkpoint]

12: // [Stage II]: distill monitors (actors frozen)
13: for  $k = 1$  to  $N_{II}$  do
14:    $\mathcal{B} \leftarrow \text{ROLLOUT}((\sigma, \_, \_, s') \mapsto \delta(\sigma, s'))$ ; Temporal Label Smoothing (Algorithm 2):  $\sigma_{t+1} \rightarrow \hat{\sigma}_{t+1}$ 
15:   update  $\{\lambda_\sigma\}$  for  $N_\lambda$  epochs with cross-entropy loss to imitate  $(o_t, a_t) \mapsto \hat{\sigma}_{t+1}$ 
16: end for

17: // [Stage III]: refine actors under learned monitors ( $\lambda$  frozen)
18: for  $k = 1$  to  $N_{III}$  do
19:    $\mathcal{B} \leftarrow \text{ROLLOUT}((\sigma, o, a, \_) \mapsto \arg \max_{\sigma'} \lambda_{\sigma'}(o, a))$ ; compute GAE
20:   update  $V$  for  $N_V$  epochs, then update active actors for  $N_\pi$  epochs; track best-return checkpoint
21: end for; [load best checkpoint]
22: return  $\Pi_{\mathcal{D}} = \{(\pi_\sigma, \lambda_\sigma)\}$ 

```

Algorithm 2 Temporal Label Smoothing ([Stage II] monitor targets)

Input: one-hot next-mode sequence $Y \in \{0, 1\}^{T \times N}$ (row $Y_{t,:} = \text{onehot}(\sigma_{t+1})$)

Parameters: kernel width $w = 7$, scale $f_s = 0.9$

```

1:  $\tau \leftarrow \text{linspace}(-3, 3, w)$ ;  $g_i \leftarrow \exp(-\frac{1}{2}(\tau_i/f_s)^2)$  ▷ 1-D Gaussian Kernel, length  $w$ 
2: for each mode  $k \in \{1, \dots, N\}$  do
3:    $\tilde{Y}_{:,k} \leftarrow g * Y_{:,k}$  ▷ 1-D convolution along time, 'same' padding
4: end for
5:  $\hat{Y}_{t,:} \leftarrow \tilde{Y}_{t,:} / \sum_k \tilde{Y}_{t,k} \quad \forall t$  ▷ renormalize distribution over modes
6: return smoothed targets  $\hat{Y}$  (each row  $\hat{Y}_{t,:} \in \Delta(\Sigma)$ ) ▷ cross-entropy targets  $\hat{\sigma}_{t+1}$ 

```

Hyperparameter		point-to-disk	RF-DMC
<u>Architecture (actor / critic / monitor MLPs)</u>			
	depth (# hidden layers)	3 / 3 / 3	4 / 4 / 4
	width (hidden dim)	64 / 256 / 64	512 / 512 / 512
	activation	SiLU	
	critic value head	two-hot, $[-20, 20]$, 255 bins	
<u>PPO Parameters</u>			
E	parallel environments	32,768	8,192
S	rollout steps	16	24
S_d	monitor distill rollout steps	32	32
M	minibatch size	4,096	1,024
$N_V / N_\pi / N_\lambda$	critic / actor / monitor epochs	3 / 1 / 1	5 / 2 / 3
γ	reward discount factor	0.99	
λ	GAE parameter	0.95	
ϵ_{clip}	PPO clip ratio	0.2	
c_{ent}	entropy coefficient	10^{-3}	
<u>Stage Budgets</u>			
N_I	[Stage I] (train)	100	500
N_{II}	[Stage II] (distill monitor)	100	200
N_{III}	[Stage III] (refine)	100	200
<u>Optimization</u>			
	optimizer	AdamW (Nesterov Momentum)	
	betas	(0.9, 0.999)	
	learning rate	5×10^{-4}	
	weight decay (weight matrices only)	10^{-4}	
	gradient clip (global norm)	10.0	
	actor schedule (warmup / cooloff steps)	linear 1000 / cosine 1000	
	critic schedule (warmup steps)	linear 1000	
	monitor schedule (warmup steps)	linear 100	
<u>Monitor Label Smoothing</u>			
	Gaussian kernel width	7	
	Gaussian kernel scale f_s	0.9	

Table 1: MSC-PPO hyperparameters across tasks.

A.2 Prompts

Refine Design We base our system prompt for design refinement on ShinkaEvolve’s [18] diff generation template – where an LLM is prompted to generate targeted code-diffs to the current solution. The system message for **Refine Design** (Snippet 1) includes user provided task & cost specification, objective, and design template (e.g. Snippet 8). Then as a user message, LEMCA supplies: a current design $\mathcal{D}$ (Snippet 4), list of inspirations (Snippet 5), design log, and corresponding metrics. The LLM responds with refinements to the current design $\mathcal{D}$ based on an “improve” (Snippet 2) or “explore” (Snippet 3) objective (see Algorithm 3). The proposed diff is parsed and applied to the current design $\mathcal{D}$ to generate a new design $\mathcal{D}_{\text{new}}$. The new design code undergoes validation – where all code paths are tested with mock inputs – to ensure it is well-formed. If validation fails, the LLM is prompted again with an error stack trace to fix the proposed code (Snippet 6).

Compact Summary LEMCA maintains a design log to track a compact summary of the design process. The design log is updated every generation after obtaining feedback metrics on the new designs. The system message for **Compact Summary** is provided in Snippet 7. The LLM is prompted to maintain a concise summary of the strategies explored, Pareto trade-offs observed, and open directions to explore, to guide the overall design process.

You are an expert control system designer tasked with designing efficient controllers that are functionally performant. Drawing inspiration from event-triggered, mode-switching, and hybrid control literature, you can propose mode-switching controller designs in the form of a Finite State Machine (FSM) that specifies the modes of operation and the state-based transition functions between the various control modes. Additionally, for each mode you should specify its sensing/compute resource configuration. The mode configuration determines the resource consumption of the controller in that mode and effectively defines the observation space for both the **control** and **monitor** policies. The control policy is trained to maximize the overall task performance through Reinforcement Learning or Model Predictive Control. Meanwhile, the monitor policy is trained to closely imitate the mode transitions determined by privileged task state information.

Your objective as a designer is to synthesize designs that achieve high task performance, while minimizing the resource costs associated with sensor acquisition and post-processing computation. By using mode-specific sensing/compute strategies, you could potentially minimize the overall resource consumption.

Mode-Switching Controller Design

To propose an FSM associated with a mode-switching controller, you should specify:

- (a) Set of operating modes, with the following attributes:
 - unique mode identifier (integer, starting from 0)
 - sensing/compute resource configuration
 - This configuration determines the observation space for the control and monitor policies in that mode, and the associated resource cost.
 - The control policy uses the mode-specific observation space to synthesize actions, while the monitor policy uses the same observation space to classify when an outgoing mode transition should fire.
- (b) Set of transitions between operating modes, with the following attributes:
 - source mode identifier
 - target mode identifier
 - Python function with signature ``Callable[[TaskState], bool]``.
 - ``TaskState`` is a task-specific type, defined in the Task Description, that represents the task's privileged state.
 - The function takes in the current task state as input and determines if the FSM should transition from source mode to target mode.
 - The FSM evaluates all the transition predicates at every step and commits to the first valid transition in the order specified.
- (c) Initial Mode, a Python function with signature ``Callable[[TaskState], int]``.
 - The function takes in the task state as input and determines the identifier of the initial mode.

The template to specify the design is as follows:

```
```python
{TEMPLATE}
```
```

Task Description

```
{TASK}
```

Resource Specification Guidelines

```
{SENSOR_SPEC}
```

```
{CONTROL_SYNT}
```

Input Format

As part of an evolutionary design loop, you will be provided with a **current design** in the population pool to modify along with **design inspirations**.

The inspirations may include a running summary of design ideas, a selection of evaluated peer designs, or both. Use them as signals about what regions of the design space have been probed, not as templates to copy from. With each design you are provided with associated evaluation metrics in a ``json`` format. Here is the schema:

```
```json
{
 "task_performance": "Expected task performance (float, or tuple of floats
 compared elementwise against the target)",
 "resource_cost": "Expected resource cost (float)",
 "task_performance_std": "Standard deviation of task performance (float)",
 "resource_cost_std": "Standard deviation of resource cost (float)",
 {ADDITIONAL_METRICS}
}
```
```

The fields ``task_performance_std`` and ``resource_cost_std`` provide evaluation variance estimates over `{N_ROLLOUTS}` rollouts. Prefer designs that meet the target performance requirements within one standard deviation.

Objective
{OBJECTIVE}

Response Format
Structure your response to contain the tags `<NAME></NAME>`, `<DESCRIPTION></DESCRIPTION>`, `<CODE></CODE>` that will be parsed afterwards.

`<NAME>`
Provide a shortened identifier (< 5 words). Lowercase, no spaces, underscores allowed.
`</NAME>`

`<DESCRIPTION>`
Provide a brief rationale (< 3 sentences) for the proposed change.
`</DESCRIPTION>`

`<CODE>`
```diff  
<<<<<< SEARCH  
# code segment in **current design** (must match exactly including  
indentation)  
=====  
# replacement code  
>>>>>> REPLACE  
<<<<<< SEARCH  
# another code segment in **current design**  
=====  
# replacement code  
>>>>>> REPLACE  
...  
`</CODE>`

**\*\*NOTE\*\*:**

- You are allowed to modify regions between "EVOLVE-BLOCK-START" and "EVOLVE-BLOCK-END". Everything outside those markers is strictly read-only.
- Use the SEARCH/REPLACE diff to propose targeted edits to the **current design**.
- Preserve the "EVOLVE-BLOCK-START" and "EVOLVE-BLOCK-END" markers. Do not introduce additional marker pairs.
- Every block's SEARCH section must be copied **verbatim** from the **current design**, including indentation.

- You can propose multiple independent edits. The SEARCH/REPLACE blocks are applied sequentially (one after another) to the **current design**. Make sure the design remains valid after the changes are applied.
- Do **not** add any other text between SEARCH/REPLACE blocks to ensure the ``diff`` can be parsed correctly.

### Improve Objective (Refine Design)

[Snippet 2]

**Synthesize Improved Design**

\* Your goal is to minimize the ``resource_cost`` while trying to achieve a ``task_performance``  $\geq$  {TARGET\_PERF}.

### Explore Objective (Refine Design)

[Snippet 3]

**Synthesize Novel Design**

\* While your primary goal remains to minimize ``resource_cost`` subject to achieving ``task_performance``  $\geq$  {TARGET\_PERF}, in this iteration prioritize exploring novel design ideas that may pay off after further refinement.

\* Be creative and propose a novel design idea that differs substantively from the current design's sensing strategy or FSM structure. Avoid purely cosmetic changes.

### User Msg (Refine Design)

[Snippet 4]

```
Current Design
```python
{design}
```

Metrics
```json
{perf}
```

Design Inspiration
{inspiration}
{design_log}
```

### Inspiration (Refine Design)

[Snippet 5]

```
Probed Design: {id}
```python
{design}
```

Metrics
```json
{perf}
```
```

### Validation Retry (Refine Design)

[Snippet 6]

Design validation failed. Encountered error: {message}.

\* Please resolve the error in the proposed code and respond in the same format keeping the `<NAME></NAME>`, `<DESCRIPTION></DESCRIPTION>`, `<CODE></CODE>` tags.

\* Ensure that the content inside NAME and DESCRIPTION tags primarily capture the essence of the proposed design, they will be parsed and tracked with the CODE.

\* The ``diff`` proposed in the CODE should still be with respect to the **current design** specified by the user.

You are an expert control system designer tasked with designing efficient controllers that are functionally performant. Your current task is to maintain a meticulous log of the various ideas explored during the design process. This design log serves as a comprehensive record of the design evolution and the rationale behind each design choice. Given the current design log and list of evaluated candidates, your task is to synthesize a compact summary that captures the key ideas and the performance-resource trade-offs. This summary will be used to inform the design of future controllers and guide effective exploration of the design space.

#### # Task Description

You are designing a controller for the following task:

{TASK}

#### # Input Format

The user provides a **current design log** and a list of **evaluated design candidates**.

With each design you are provided with associated evaluation metrics in a ``json`` format. Here is the schema:

```
```json
{
  "task_performance": "Expected task performance (float, or tuple of floats compared elementwise against the target)",
  "resource_cost": "Expected resource cost (float)",
  "task_performance_std": "Standard deviation of task performance (float)",
  "resource_cost_std": "Standard deviation of resource cost (float)",
  {ADDITIONAL_METRICS}
}
```

The fields ``task_performance_std`` and ``resource_cost_std`` provide evaluation variance estimates. Prefer designs that meet the target performance requirements within one standard deviation.

Objective

Synthesize Improved Design

* Your goal is to minimize the ``resource_cost`` while trying to achieve a ``task_performance`` $\geq$ {TARGET_PERF}.

Response Format

Provide an updated condensed summary of the design log. Make sure you capture the key ideas explored along with the resource-performance trade-offs to guide future design iterations.

Structure your response inside `<LOG></LOG>` tags.

```
<LOG>
```markdown
Strategies Explored
<!-- One bullet per distinct design direction. Include best observed (perf, cost). Merge duplicates; drop strategies dominated by newer evidence. -->
Pareto Trade-offs
<!-- Characterize the pareto frontier. Which axes move performance? Which move cost? Where do they couple? -->
Open Directions
```

```

<!-- Unexplored or under-explored regions of the design space; hypotheses
worth testing next. Rank by potential payoff and novelty. -->
...

</LOG>

NOTE:
* Keep the design log concise. The total log should be under 1000 words.
* When new candidates are presented, merge them into existing bullet points
rather than simply appending. Prune any bullet point that is dominated.

```

492

#### Design Template (point-to-disk)

[Snippet 8]

```

from typing import Any, Sequence, Callable, Tuple
import jax
import jax.numpy as jnp
from lemca.tasks.p2d.base import DesignABC, GPS, SectorRadar

class Design(DesignABC):
 @property
 def mode_obs(self) -> dict[int, GPS | SectorRadar | Sequence[GPS |
SectorRadar]]:
 ## EVOLVE-BLOCK-START
 return {}
 ## EVOLVE-BLOCK-END

 @property
 def mode_transitions(self) -> Sequence[Tuple[int, int, Callable[[Any],
bool]]]:
 ## EVOLVE-BLOCK-START
 return []
 ## EVOLVE-BLOCK-END

 @property
 def initial_mode(self):
 ## EVOLVE-BLOCK-START
 def f(mdp_state: Any) -> int:
 return 0
 return f
 ## EVOLVE-BLOCK-END

```

493

### A.3 LEMCA : Designer Loop

LEMCA utilizes a generic genetic algorithm (GA) loop that utilizes a feasible-first [22] tournament selection [23, 24]. The crossover operator is handled by the LLM by suitable RefineDesign prompts (outlined in Appendix A.2). The refinement process starts with a few monolithic seed designs that characterize the trade-offs involved in monolithic configurations. The population size for the GA loop is capped to  $N$  designs: all  $N_{\text{new}}$  offspring and top  $N - N_{\text{new}}$  designs from the previous generation are retained. The overall design loop of LEMCA is described in Algorithm 3. Across tasks, we use a common set of hyperparameters for the design loop and they are outlined in Table 2. For all our experiments we use the gemini-3.1-pro reasoning LLM from Google DeepMind for its strong instruction following and code generation capabilities. We use the generic “chat completion” method exposed in the any-llm library [49] to interface with the Gemini API, and resort to default sampling parameters (e.g. temperature is set to 1.0) – we leave optimizing prompts, sampling, and model selection for future work.

---

**Algorithm 3** LEMCA: LLM-Guided synthesis of Efficient Mode-Switching Control Architectures

---

**Input:** MDP  $\mathcal{M}$ , Sensor Library  $\Phi$ , Cost  $C : \Phi \rightarrow \mathbb{R}^+$ , Synthesis Procedure  $M$ , Seed Population:  $D$

**Parameters:**

$N$  (population size),  $G$  (generations),  $N_{\text{new}}$  (#offspring),  $T$  (tournament size),  
 $\epsilon$  (performance threshold),  $K$  (#inspirations),  $\rho$  (explore rate)

```
1: procedure SELECT(D) ▷ Feasible-first Rule [22]
2: Sample T designs from D ; rank by $(\max(\epsilon - J(\mathcal{D}), 0), R(\mathcal{D}))$; return best.
3: end procedure

4: procedure REFINEDSIGN($\mathcal{D}_{\text{curr}}, \mathcal{D}_{\text{insp}}, L$, objective)
5: prompt $\leftarrow$ sys_msg($\mathcal{M}, \Phi, C$, objective) ▷ Snippets 1 to 3
6: prompt $\leftarrow$ prompt + user_msg($\mathcal{D}_{\text{curr}}, \mathcal{D}_{\text{insp}}, L$) ▷ Snippets 4 and 5
7: retry $\leftarrow 0$
8: while retry $<$ max_retries do
9: $\mathcal{D}_{\text{new}} \leftarrow$ LLM(prompt)
10: if isValid($\mathcal{D}_{\text{new}}$) then
11: return $\mathcal{D}_{\text{new}}$
12: else
13: prompt $\leftarrow$ prompt + validation_msg(error)
14: retry $\leftarrow$ retry + 1
15: end if
16: end while
17: end procedure

18: procedure COMPACTSUMMARY($L, \mathcal{D}_{\text{new}}$)
19: prompt $\leftarrow$ compact_sys_msg($\mathcal{M}, \Phi, C$) + compact_user_msg($L, \mathcal{D}_{\text{new}}$) ▷ Snippet 7
20: $L_{\text{new}} \leftarrow$ LLM(prompt)
21: return L_{new}
22: end procedure
```

---

```
 // LEMCA Main Loop
23: $D_0 \leftarrow D$
24: $L \leftarrow$ COMPACTSUMMARY($\emptyset, D_0$)
25: for $g = 1$ to G do
26: $D_{\text{new}} \leftarrow \emptyset$
27: // Design Generation
28: for $n = 1$ to N_{new} do
29: $\mathcal{D} \leftarrow$ run SELECT(D_{g-1}) until $K + 1$ distinct designs are selected.
30: $\mathcal{D}_{\text{curr}}, * \mathcal{D}_{\text{insp}} \leftarrow \mathcal{D}$
31: objective $\leftarrow$ “explore” if rand() $<$ ρ else “improve”
32: $\mathcal{D}_{\text{new}} \leftarrow$ REFINEDSIGN($\mathcal{D}_{\text{curr}}, \mathcal{D}_{\text{insp}}, L$, objective)
33: $D_{\text{new}} \leftarrow D_{\text{new}} \cup \{\mathcal{D}_{\text{new}}\}$
34: end for
35: // Synthesis + Evaluation
36: for $\mathcal{D}_i \in D_{\text{new}}$ do
37: $\Pi_{\mathcal{D}_i} \leftarrow M(\mathcal{D}_i)$; estimate $R(\mathcal{D}_i)$ and $J(\mathcal{D}_i)$
38: end for
39: // Update Design Log
40: $L \leftarrow$ COMPACTSUMMARY(L, D_{new})
41: // Update Active Population and Design Archive
42: $\mathcal{D} \leftarrow$ select top $(N - N_{\text{new}})$ designs from D_{g-1} ordered by $(\max(\epsilon - J(\mathcal{D}), 0), R(\mathcal{D}))$
43: $D_g \leftarrow \mathcal{D} \cup D_{\text{new}}$; $D \leftarrow D \cup D_{\text{new}}$
44: end for
45: return $\mathcal{D}_{\epsilon}^* = \arg \min_{\mathcal{D} \in D} (\max(\epsilon - J(\mathcal{D}), 0), R(\mathcal{D}))$
```

---

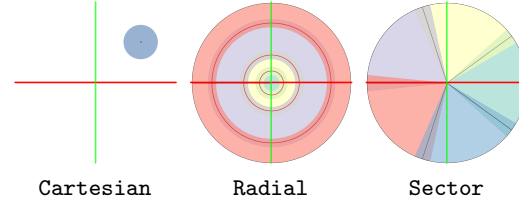
Hyperparameters		Value
$N$	Population Size	20
$N_{\text{new}}$	Offspring per Generation	3
$T$	Tournament Size	5
$K$	Number of Inspirations	3
$\rho$	Explore Rate	0.2
	max_retries (LLM Validation)	1

Table 2: The hyperparameters of LEMCA designer loop across tasks.

## 507 B point-to-disk

508 **Task** A velocity-controlled point-mass with state  $\mathbf{x}$  (cartesian position) must be driven into a target  
509 disk of radius  $r^* = 0.01$  at the origin and held there. Given a velocity command  $\mathbf{u}$ , the system evolves  
510 as  $\dot{\mathbf{x}} = \mathbf{v}$  with  $\mathbf{v} \sim B_\epsilon(\mathbf{u}) := \{\mathbf{v} : \|\mathbf{v} - \mathbf{u}\|_2 \leq \epsilon \|\mathbf{u}\|_2\}$  and  $\epsilon = 0.25$  – the actuation noise grows with  
511 the commanded speed, so slowing down ( $\mathbf{u} \rightarrow \mathbf{0}$ ) allows the controller to suppress actuation noise.  
512 The controller, operating at 50 Hz, is scored by its dwell time inside the target for rollouts of length  
513  $H = 500$  steps (10 s) from states on the unit circle,  $J(\pi) = \mathbb{E}_{\pi, \{\mathbf{x}_0 : \|\mathbf{x}_0\|_2 = 1\}} \left[ \sum_{t=0}^H \mathbb{1}_{[\|\mathbf{x}_t\|_2 < r^*]} \right]$ .  
514 The designer’s goal is to meet a performance target  $J_{\text{target}}$  while minimizing the sensing cost incurred  
515 en route; the sensor library and cost model used in the experiments (Sec. 3) are detailed below.

516 **Sensors** The designer can equip each mode  
517 with one or more of three sensor types, each  
518 revealing a different slice of the robot’s position  
519  $\mathbf{x}$ . In each mode, the agent’s observation is the  
520 concatenation of the active sensors’ readings  
521 (optionally stacked over a history length  $h$ ), and  
522 pays the sum of their per-step costs.



- 523 • **Cartesian (GPS)**: noisy position  $\mathbf{x} + \mathcal{N}(0, \sigma_{xy}^2 \mathbf{I}_2)$ , with  $\sigma_{xy} \in \{0, 10^{-2}, 5 \cdot 10^{-2}, 10^{-1}, 5 \cdot 10^{-1}\}$ .
- 524 • **Radial & Sector (RadialSector)**: a discretized polar reading returning the active sector among  
525  $s \in [1, 360]$  equiangular sectors (optionally offset by  $\phi$  radians) and the active radial band  
526 among  $\text{len}(\mathbf{d})+1$  bands defined by increasing distance thresholds  $\mathbf{d} = (d_1, \dots) \leq 1$ . The  
527 per-step output is a concatenation of the one-hot over radial bands (empty when  $\mathbf{d} = ()$ ) and  
528 unit-vector  $(\cos \theta_s, \sin \theta_s)$  corresponding to centroid of the active sector. There is some Gaussian  
529 noise in measuring the polar coordinates of the robot  $(r, \theta)$ . The noise scales are tunable with  
530  $\sigma_\theta \in \{0, 10^{-1}, 2 \cdot 10^{-1}, 4 \cdot 10^{-1}, 8 \cdot 10^{-1}\}$  and  $\sigma_r \in \{0, 10^{-2}, 5 \cdot 10^{-2}, 10^{-1}\}$ .

### 531 B.1 Cost Model

532 We adopt an information-theoretic per-step cost that prices a sensor configuration  $\varphi$  by the number of  
533 bits it resolves about the state. For a quantity confined to a range “span” and recovered with effective  
534 standard deviation “ $\sigma$ ”, we use the resolution heuristic:  $\text{bits}(\text{span}, \sigma) \approx \max(\log_2(\text{span}/\sigma), 0)$ , i.e.  
535 the bits needed to index the  $\approx \text{span}/\sigma$  distinguishable  $\sigma$ -wide cells tiling the range. The per-step cost  
536 includes a fixed activation term, a term that scales with the bits resolved along each measured axis,  
537 and a storage term as follows:

$$C_{\text{radar}}(\varphi) = c_{\text{base}}^{\text{rad}} + c_{\text{ang}} \cdot \text{bits}(2\pi, \tilde{\sigma}_\theta) + c_{\text{dist}} \cdot \text{bits}(r_{\text{max}}, \tilde{\sigma}_r) + \underbrace{c_{\text{mem}} \log_2 h}_{\text{storage}}$$

$$C_{\text{gps}}(\varphi) = c_{\text{base}}^{\text{gps}} + 2 c_{\text{gps}} \cdot \text{bits}(2r_{\text{max}}, \sigma_{xy}) + c_{\text{mem}} \log_2 h,$$

538 where the GPS factor of  $2c_{\text{gps}}$  accounts for the two cartesian axes (each spanning  $[-r_{\text{max}}, r_{\text{max}}]$ ).

**Effective noise from quantization** A `RadialSector` resolves the state through both its measurement noise *and* its discretization, so we convert each discretization into an equivalent variance and add it to the measurement variance before estimating the bits:  $\tilde{\sigma} = \sqrt{\sigma_{\text{quant}}^2 + \sigma_{\text{meas}}^2}$ . For the *angular* channel, knowing only which of the  $s$  equiangular sectors contains the heading vector results in a uniform spread of possible locations over an arc of width  $w = 2\pi/s$ . This spread is measured by the mean chord length  $R = \sin(w/2)/(w/2)$ , with  $R=1$  a perfectly localized heading and  $R=0$  presenting no information about the heading. This results in an equivalent quantization variance  $\sigma_{\text{quant},\theta}^2 = -2 \ln R$ , which is added to the measurement variance  $\sigma_\theta^2$  to get the effective noise  $\tilde{\sigma}_\theta$  for the angular channel. For the *radial* channel, bands with edges  $0 = d_0 < d_1 < \dots < d_n < d_{n+1} = r_{\text{max}}$  localize the radial distance only to the band it falls in; a uniform spread across a band of width  $w_i = d_i - d_{i-1}$  contributes variance  $w_i^2/12$ . Weighting each band by the mass an annulus receives under a spatially-uniform prior on the disk,  $p_i = (d_i^2 - d_{i-1}^2)/r_{\text{max}}^2$ , gives  $\sigma_{\text{quant},r}^2 = \sum_i p_i w_i^2/12$  and  $\tilde{\sigma}_r = \sqrt{\sigma_{\text{quant},r}^2 + \sigma_r^2}$ . With  $\mathbf{d} = ()$ , the radial channel is uninformative and its cost is dropped. Similarly, with  $s = 1$ , the angular channel becomes uninformative and is dropped from the cost. This cost specification exposes a smooth, monotone cost function over the parameter space of Sensor configurations, which the designer can navigate to trade off cost and performance.

**History as storage cost, not re-acquisition** A mode that stacks  $h$  readings already pays the full per-step acquisition cost (the bits above) at *every* step, so buffering past observations reveals no information the sensor stream has not already been charged for. Its only marginal cost is the memory to retain  $h$  readings, which we price as the sub-linear  $c_{\text{mem}} \log_2 h$  – reflecting the addressing/storage overhead of a depth- $h$  buffer rather than charging  $h$ -fold the acquisition cost.

**Cost Structures** We instantiate this model with two coefficient sets that encode contrasting hardware regimes – `CostlyGPS` and `CheapGPS`. Both share  $r_{\text{max}} = 2$  and  $c_{\text{mem}} = 0.005$ .

Cost Structure	GPS		RadialSector		
	$c_{\text{base}}^{\text{gps}}$	$c_{\text{gps}}$	$c_{\text{base}}^{\text{rad}}$	$c_{\text{ang}}$	$c_{\text{dist}}$
CostlyGPS	0.01	0.01	0.01	0.01	0.01
CheapGPS	0.02	0.01	0.01	0.05	0.01

Under `CostlyGPS`, the radial and angular bits are cheap, so `RadialSector` sensors dominate the affordable end of the spectrum. In `CheapGPS`, the angular coefficient is increased by  $5\times$ , making well-chosen Cartesian configurations cost-competitive with `RadialSector`. Table 3 reports representative per-step costs under each structure.

Sensor configuration	CostlyGPS	CheapGPS
<code>SectorRadar(s=1, d=())</code>	0.010	0.010
<code>SectorRadar(s=3, d=())</code>	0.044	0.178
<code>SectorRadar(s=4, d=(0.01,0.05))</code>	0.066	0.217
<code>SectorRadar(s=8, d=(0.05,0.2,0.5))</code>	0.080	0.272
<code>GPS(sigma_xy=0.01)</code>	0.183	0.193
<code>GPS(sigma_xy=0.05)</code>	0.136	0.146
<code>GPS(sigma_xy=0.1)</code>	0.116	0.126

Table 3: Representative per-step sensor costs under the two cost structures.

## B.2 Experimental Setup

A design  $\mathcal{D}$  is a Python program specifying `mode_obs` (the sensor configuration per mode), `mode_transitions` (predicates over the position  $\mathbf{x}$ ), and `initial_mode` (see the template in Snippet 8). Each candidate is synthesized by running MSC-PP0 (Appendix A.1, hyperparameters in Table 1)

with 3 random seeds; the privileged critic and monitors operate on the polar state ( $\|\mathbf{x}\|, \cos \theta, \sin \theta$ ). The oracle controller with perfect state achieves  $J_{\text{oracle}} = 456.23 \pm 4.59$  ( $N = 1000$ ). We set performance targets for LEMCA to  $J_{\text{target}} \in \{200, 300, 450\}$  ( $\approx 0.44, 0.66, 0.98$  of oracle performance), spanning aggressive to near-lossless performance budgets.

We seed the search with 10 monolithic configurations (see Table 4) – containing 7 `RadialSector` configurations spanning sector counts and angular-noise levels and 3 `Cartesian` configurations – and run LEMCA for 8 generations under each (cost structure, performance target) pair. These seed designs trace out an approximate monolithic frontier, and the same seed population is used for both the cost structures. LEMCA is provided with the code implementing the task and cost function as part of the system prompt.

Configuration	J/J <sub>oracle</sub>	Per-step cost	
		CostlyGPS	CheapGPS
<u>RadialSector monoliths</u>			
SectorRadar(s=3, sigma_theta=0.8)	0.39 ± 0.03	0.036	0.142
SectorRadar(s=3)	0.43 ± 0.03	0.044	0.178
SectorRadar(s=4)	0.66 ± 0.03	0.048	0.199
SectorRadar(s=5, sigma_theta=0.2)	0.76 ± 0.02	0.049	0.206
SectorRadar(s=7)	0.80 ± 0.02	0.056	0.240
SectorRadar(s=10, sigma_theta=0.2)	0.83 ± 0.02	0.055	0.237
SectorRadar(s=360)	0.87 ± 0.01	0.113	0.524
<u>Cartesian monoliths</u>			
GPS(sigma_xy=0.05, history=5)	0.49 ± 0.08	0.148	0.158
GPS(sigma_xy=0.01)	0.98 ± 0.01	0.183	0.193
GPS(sigma_xy=10 <sup>-6</sup> )	1.00 ± 0.01	0.449	0.459

Table 4: The monolithic seed designs used to initialize LEMCA for point-to-disk.

## C RF-DMC (RangeFinder Variants of `dm_control`)

We construct rangefinder variants of three `dm_control` [11, 25] tasks – `cartpole-swingup`, `cup-catch`, and `finger-spin`. In each, the agent observes the world through a `RayScanSensor` consisting of 360° rangefinders that are rigidly attached to a task-relevant body site (pole tip, cup center, fingertip) and cast equiangular rays that report the distance to the nearest enclosing-box surface (we wrap each scene in walls/ceiling so every ray returns a finite range). For each mode, the designer configures a `RayScanSensor` by choosing its measurement quality  $q \in \{A, \dots, F\}$ , the number of equispaced rays  $n$  (a divisor of 360), an optional angular offset  $\phi$ , and a history length  $h$ . The mode-transition predicates  $\delta$  and initial-mode map  $\nu$  are privileged: they read interpretable simulator state (e.g. `pole_angle`, `ball_to_target`, `hinge_angvel`) and these transitions are distilled into rangefinder-based monitors by MSC-PP0 (Appendix A.1). Task performance is the oracle-normalized episodic return,  $J/J_{\text{oracle}}$ , estimated over  $N=1000$  rollouts.

**Tasks** All three tasks run for  $H = 1000$  steps with a per-step reward in  $[0, 1]$ .

- `cartpole-swingup` (100 Hz): a force-controlled cart must swing up a hinged pole and balance it upright near the center of the rail. The reward multiplicatively combines uprightness, centering, small control, and small angular velocity. To ensure the task is solvable by MSC-PP0 we use a wider reset distribution during training by spawning the pole roughly upright 50% of the time, but preserve the standard reset distribution during evaluation.
- `cup-catch` (50 Hz): a force-actuated cup must catch a ball tethered to it by a 0.3 m rope, possibly swinging it up first. The reward is the indicator function checking if the ball is successfully caught.
- `finger-spin` (50 Hz): a two-link finger must repeatedly strike a free-hinged spinner to maintain an angular velocity ( $> 15$  rad/s). The reward is the indicator function checking if the spinner’s

angular velocity exceeds the target. For MSC-PP0 training we use a smoother reward function that provides a dense signal to get the spinner spinning at a rate higher than the target. However, the original binary reward is preserved during controller evaluation.

Table 5 summarizes the per-task settings, oracle performance, and the performance target used for LEMCA in the experiments presented in Sec. 3.

Task	Rangefinder Site	Priv. State Size	$J_{\text{oracle}}$	$J_{\text{target}} (/J_{\text{oracle}})$
carpole-swingup	pole tip	5	$881.8 \pm 4.9$	650 (0.74)
cup-catch	cup center	10	$977.8 \pm 15.7$	700 (0.72)
finger-spin	finger tip	19	$969.9 \pm 11.7$	750 (0.77)

Table 5: RF-DMC task settings. The oracle performance corresponds to episodic return achieved with full state information (measured over  $N = 1000$  rollouts).

### C.1 Rangefinder Sensor Model

We adopt the probabilistic beam-sensor model of Thrun et al. [26], a well-characterized non-Gaussian noise model that abstracts noisy exteroceptive sensors (e.g. LiDAR) while remaining fast to simulate. A measured range  $z$  given the true range  $z^*$  is drawn from a four-component mixture capturing: measurement noise, unexpected short returns, detection failures, and spurious returns.

$$p(z | z^*) = c_{\text{hit}} p_{\text{hit}}(z) + c_{\text{short}} p_{\text{short}}(z) + c_{\text{max}} p_{\text{max}}(z) + c_{\text{rand}} p_{\text{rand}}(z), \quad \sum_* c_* = 1,$$

Where:

- $p_{\text{hit}} = \mathcal{N}_{[z_{\min}, z_{\max}]}(z^*, \sigma_{\text{hit}}^2)$  (truncated Gaussian), models the inherent measurement noise.
- $p_{\text{short}} = \text{Exp}_{[z_{\min}, z^*]}(\lambda_{\text{short}})$  (truncated exponential), models unexpected interference.
- $p_{\text{max}} = \delta_{z=z_{\max}}$ , models detection failures.
- $p_{\text{rand}} = \text{Uniform}(z_{\min}, z_{\max})$ , models spurious readings.

Each measurement is quantized to resolution  $z_{\text{res}}$  and normalized to  $[0, 1]$  by  $z_{\text{max}}$ . The discrete quality grades A (cleanest) through F (noisiest) instantiate distinct mixture weights and  $\sigma_{\text{hit}}$  (all share  $c_{\text{short}} = 0$ ,  $z_{\text{res}} = 10^{-5}$ ,  $z_{\text{max}} = 10$ ), and are summarized in Table 6.

Each grade’s per-measurement energy is derived from its 90th-percentile signal-to-noise ratio  $\text{SNR}_{\text{dB}}$  over the sensor’s operating range  $[z_{\min}, z_{\max}]$  (estimated by Monte-Carlo sampling of the beam model) as:

$$E(q) = k_s z_{\text{max}}^4 \text{SNR}_{\text{dB}}^3 + k_c z_{\text{res}}^{-1}, \quad k_s = 10^{-2}, \quad k_c = 10^{-3},$$

where the first term models acquisition energy (higher quality requires substantially more energy) and the second a fixed representation/compute overhead. Table 6 reports these energies normalized to grade F, which serve as the per-ray costs in the cost model described below.

Grade	$c_{\text{hit}}$	$c_{\text{max}}$	$c_{\text{rand}}$	$\sigma_{\text{hit}}$	$\text{SNR}_{\text{dB}}$	Rel. Cost / Ray
A	0.995	0.003	0.002	$10^{-5}$	32.0	22.61
B	0.990	0.005	0.005	$8 \cdot 10^{-4}$	28.1	15.39
C	0.970	0.010	0.020	$10^{-3}$	22.2	7.62
D	0.950	0.010	0.040	$10^{-2}$	19.4	5.08
E	0.850	0.050	0.100	$5 \cdot 10^{-2}$	15.2	2.44
F	0.650	0.100	0.250	$10^{-1}$	11.3	1.00

Table 6: Rangefinder quality grades: mixture weights, hit-noise scale, 90th-percentile SNR, and per-ray energy cost normalized to grade F (all grades use  $c_{\text{short}}=0$ ).

## 626 C.2 Cost Model

627 The per-step cost of a RayScanSensor scales the per-ray energy by the number of rays and adds a  
628 logarithmic memory term for the history buffer:

$$C(\varphi) = (E(q) + c_{\text{mem}} \log_2 h) \cdot n, \quad c_{\text{mem}} = 0.5,$$

629 with  $E(q)$  the normalized per-ray cost from Table 6. This couples the three knobs the way physical  
630 scanning rangefinders behave: cost grows linearly in resolution  $n$ , increases with quality through  
631  $E(q)$ , and pays only a sub-linear ( $\log_2 h$ ) storage premium for buffering history (which re-uses  
632 already-acquired scans, cf. Appendix B.1). A mode with an empty sensor list has zero cost. This cost  
633 model is shared across all the three tasks.

## 634 C.3 Experimental Setup

635 A design  $\mathcal{D}$  specifies `mode_obs` (a RayScanSensor configuration per mode), `mode_transitions`  
636 (predicates over the privileged state), and `initial_mode`; each candidate is synthesized with  
637 MSC-PP0 (Appendix A.1, Table 1) using two seeds per design to lower training costs. We seed  
638 LEMCA with 11 monoliths (Table 7) and run it for 10 generations. The seed designs are obtained by  
639 selecting RayScan configurations  $(q, h, n)$  to uniformly span the cost spectrum. The seed designs  
640 provide an approximate per-task monolithic frontier – demonstrating the influence of resolution  $n$ ,  
641 quality  $q$ , and history  $h$  on performance – and the same seed population is used for all three tasks.  
642 LEMCA is provided with the code implementing the task and cost function as part of the system  
643 prompt.

Config $(q, h, n)$	cost/step	Task Performance: $J/J_{\text{oracle}}$		
		cartpole-swingup	cup-catch	finger-spin
(F, 4, 3)	6.0	0.26	0.12	0.00
(E, 8, 3)	11.8	0.60	0.11	0.09
(D, 16, 3)	21.2	0.96	0.11	0.37
(C, 4, 4)	34.5	0.84	0.82	0.65
(C, 4, 10)	86.2	0.93	0.83	0.94
(E, 8, 60)	236.6	0.54	0.22	0.61
(C, 8, 60)	546.9	0.94	0.34	0.95
(A, 4, 60)	1416.5	0.96	0.99	0.99
(B, 4, 180)	2949.7	0.92	0.98	0.99
(A, 2, 360)	8319.2	0.80	0.99	0.99
(A, 16, 360)	8859.2	0.98	0.98	0.87

Table 7: The 11 monolithic seed designs used to initialize LEMCA across all RF-DMC tasks. Each seed design uses the same sensor `RayScanSensor(quality=  $q$ , history=  $h$ , nrays=  $n$ )` throughout the task. The task performance is measured over  $N=1000$  rollouts.

## 644 D clutter-nav (Cluttered Navigation with Radar Sensors)

645 A disk-shaped robot with unicycle (differential-drive) dynamics must reach a goal while avoiding  
646 static and dynamic disk obstacles in a cluttered planar world. Unlike `point-to-disk` and RF-DMC,  
647 the MSC controllers in this task are *not* synthesized by MSC-PP0: instead each mode configures the  
648 Radar sensor and Model-Predictive Path-Integral (MPPI) [27] planner. The MPPI planner acts under  
649 a naive certainty-equivalence assumption – treating the noisy obstacle detections of the Radar as the  
650 ground truth state. Additionally, the designer synthesizes mode-transition predicates *directly* over the  
651 same noisy detections the planner sees – thus, no controller learning or monitor distillation is required  
652 for this task. This setup exercises LEMCA with a different synthesis procedure, performance metric,  
653 and a joint sensor-and-planner design space.

**Task** The robot is spawned in a  $[-15, 15]^2$  m plane. At reset, a random number of obstacles are spawned at uniformly random positions, drawing  $n_{\text{static}} \in [5, 20]$  static and  $n_{\text{dynamic}} \in [0, 15]$  dynamic obstacles with radii in  $[0.3, 2.0]$  m. The agent (radius 0.5 m) starts collision-free with a goal sampled 10–15 m away; an episode succeeds if the agent reaches within  $\text{goal\_tol} = 0.25$  m of the goal before a 25 s ( $H = 500$  steps at 20 Hz,  $dt = 0.05$  s) time limit. At each timestep, the agent commands  $(v, \omega)$  with  $v \in [0, 5]$  m/s (forward velocity) and  $\omega \in [-3, 3]$  rad/s (turning rate). There is actuation noise where the realized velocity is commanded velocity perturbed with a Gaussian noise that scales with the command magnitude ( $\sigma = (0.1, 0.1)$ ). The dynamic obstacles also operate with unicycle dynamics and follow a soft-repulsion-plus-goal-tracking policy towards periodically resampled goals. When simulating this task the collision dynamics are disabled, so the agent and obstacles can interpenetrate. However, the number of such collisions is recorded and factored into the performance metric. Task performance is measured by the pair  $J = (\text{success rate}, 1/\text{cpt}_{90})$ , where the collision rate  $\text{cpt}_{90}$  is the 90th-percentile over episodes of collisions / steps. LEMCA minimizes resource cost associated with sensor-planner configuration subject to a feasibility constraint of  $\geq 80\%$  success while pushing  $1/\text{cpt}_{90}$  above a configured target threshold (Appendix D.4). We assume the agent always has access to a compass that reports the relative goal bearing and distance accurately.

## D.1 Radar Sensor Model

Each mode’s Radar configuration returns ego-centric `Detections` – range  $r$ , bearing  $\text{az}$ , size  $\text{sz}$ , and radial velocity  $v_r$  – for obstacles within  $\text{max\_range}$  and  $|\text{az}| \leq \text{fov}/2$ . Detection reliability and noise are governed by an SNR proxy  $P = (R_0/r)^4$ : the detection probability is  $p_{\text{detect}} = p_{\text{fp}}^{1/(1+P)}$  (near-certain for  $r \ll R_0$ , decaying to the false-positive rate beyond  $R_0$ ), and noise associated with different dimensions (range, bearing, size, velocity) scale as  $\sigma_*(r) = \hat{\sigma}_* r^2/R_0$ , with the base noise scales being  $\hat{\sigma}_r = 0.2$  m,  $\hat{\sigma}_{\text{az}} = 0.05$  rad,  $\hat{\sigma}_{\text{sz}} = 0.08$  m,  $\hat{\sigma}_v = 0.3$  m/s. The detections are quantized into  $1^\circ$  angular bins and return only the nearest obstacle per bin (respecting occlusions),  $v_r$  captures only the radial component of obstacle motion. Additionally, spurious detections (that appear as random blips in the ego-centric `Detections`) arrive at  $\text{Poisson}(1)$  per scan. Thus larger  $R_0$ ,  $\text{max\_range}$ , and  $\text{fov}$  enable cleaner, longer-range, and wider coverage but come at a higher cost. Of the various knobs, the designer controls only  $(R_0, \text{max\_range}, \text{fov})$ ; the noise scales and false-positive rate are held fixed across modes.

## D.2 MPPI Planner and Certainty Equivalence

Each mode is driven by its own MPPI planner configuration acting under *certainty equivalence*: it treats the latest noisy `Detections` as the true obstacle state and does not reason about sensing uncertainty. The planner is warm-started with a shifted nominal plan, so on re-entering a mode (or right after a transition) it always refines the previous nominal plan rather than starting from scratch. In a mode, MPPI re-plans once every  $\text{period}$  steps and commits to the plan for  $\text{period} - 1$  steps, amortizing both sensing and compute costs.

A plan is parameterized by  $K = 6$  control knots per action dimension  $(v, \omega)$  in a normalized space  $[-1, 1]$ . This is lifted to the  $T = \text{horizon}/dt$ -step control sequence by a fixed cubic Catmull–Rom spline basis and is affinely mapped to the actuation limits  $v \in [0, 5]$  m/s and  $\omega \in [-3, 3]$  rad/s, yielding smooth, low-dimensional plans. Each re-plan first shifts the previous plan to seed an initial warmstarted plan, then runs  $\text{n\_iters}$  refinement iterations. At iteration  $i$  it samples  $\text{n\_samples}$  knot perturbations from a zero-mean Gaussian whose knot-space scale anneals geometrically from  $\sigma_0 = 0.8$  to  $\sigma_1 = 0.1$ ,  $\sigma(i) = \sigma_0(\sigma_1/\sigma_0)^{i/(\text{n\_iters}-1)}$  (early iterations explore broadly, later iterations do local refinements). The perturbed trajectories are rolled out and scored, to finally update the nominal knots to the soft-min weighted mean  $w_\tau \propto \exp(-(\text{cost}(\tau) - \min_{\tau'} \text{cost}(\tau'))/\lambda)$  with temperature  $\lambda = 10$ .

Each rolled-out trajectory  $\tau = \{p_t\}_{t=1}^T$  (ego-frame, starting at the origin) is scored against the ego-centric goal  $g$  and the obstacles by a weighted sum of five terms,

$$\text{cost}(\tau) = w_{\text{hit}} C_{\text{hit}} + w_{\text{soft}} C_{\text{soft}} + w_{\text{goal}} C_{\text{goal}} + w_{\text{prog}} C_{\text{prog}} + w_{\text{ctrl}} C_{\text{ctrl}},$$

Let  $d_{t,m}$  denote the clearance to obstacle  $m$  at step  $t$  (clearance measures the distance between the centers minus the agent+obstacle radii), then the five cost terms are:

- $C_{\text{goal}} = \min_t \|p_t - g\|$  — closest approach to the goal over the horizon ( $w_{\text{goal}}=5$ )
- $C_{\text{prog}} = \text{mean}_t \max(\|p_t - g\| - \rho, 0)$  — mean residual distance rewarding steady progress ( $w_{\text{prog}}=3$ )
- $C_{\text{hit}} = \sum_t \mathbb{1}_{[\min_m d_{t,m} < 0]}$  — # of horizon steps in collision ( $w_{\text{hit}}=50$ )
- $C_{\text{soft}} = \sum_t \exp(-(\max(\min_m d_{t,m}, 0)/\eta)^2)$  — a smooth proximity penalty with  $\eta=0.1$  m, disabled once the robot is 1 m from the goal ( $w_{\text{soft}}=2$ )
- $C_{\text{ctrl}} = \sum_k (v_k+1)^2 + \omega_k^2$  — knot-space control effort, referenced so  $v=0, \omega=0$  is the zero-cost minimum ( $w_{\text{ctrl}}=1$ )

When rolling out a plan, the obstacles are propagated forward with their observed radial velocity  $v_r$  under a constant-velocity model, and the clearance  $d_{t,m}$  is computed against these propagated obstacles. Of the various knobs, the designer controls only (n\_samples, n\_iters, horizon, period); everything else is held fixed across modes.

	Parameter	Description	Domain
Sensor	$R_0$	sensor power (reliable-detection range, $\propto$ SNR)	[2, 50]
	max_range	range cutoff for returned detections	[5, 15]
	fov	angular field of view (rad); $2\pi$ omnidirectional	$[\pi/3, 2\pi]$
Planner	n_samples	MPPI rollouts per refinement iteration	{256, 512, 1024}
	n_iters	MPPI refinement iterations	{1, ..., 5}
	horizon	planning horizon (s)	[2, 5]
	period	re-plan period	{1, ..., 20}

Table 8: clutter-nav per-mode Sensor + Planner Configuration space.

### D.3 Design Space and Cost Model

Each mode maps to a single Configuration jointly specifying Sensor and Planner parameters (Table 8). The per-step cost sums a sensor (energy), coverage (area), and compute term, amortized by the re-plan period:

$$C(\varphi) = \frac{1}{p} \left( \underbrace{w_s (R_0/R_{\text{ref}})^2}_{C_{\text{sensor}}} + \underbrace{w_c \frac{0.5 \max\_range^2 \text{fov}}{A_{\text{ref}}}}_{C_{\text{cov}}} + \underbrace{w_p \frac{n \cdot i \cdot \text{horizon}}{F_{\text{ref}}}}_{C_{\text{compute}}} \right),$$

With sensor power measured relative to  $R_{\text{ref}} = 2$ , coverage to the wedge  $A_{\text{ref}} = 0.5 \cdot 10^2 \cdot (\pi/3)$ , and the planner compute to the lowest rollout budget  $F_{\text{ref}} = 256 \cdot 1 \cdot 2.0$ . A larger re-plan period  $p$  amortizes both sensing and compute at the risk of acting on stale plans. We use weights  $(w_s, w_c, w_p) = (1.0, 0.5, 1.0)$  for our experiments (Sec. 3). The resource cost spans three orders of magnitude across the design space, some examples are provided in Table 9.

Configuration	cost/step
(R0=2, max_range=10, fov= $\pi/3$ , n_iters=1, n_samples=256, horizon=2, period=10)	0.25
(R0=20, max_range=10, fov= $2\pi$ , n_iters=3, n_samples=512, horizon=3, period=4)	28.00
(R0=50, max_range=15, fov= $2\pi$ , n_iters=5, n_samples=1024, horizon=5, period=1)	681.75

Table 9: Example per-step costs for clutter-nav under cost model described in Appendix D.3.

### D.4 Experimental Setup

The monolithic frontier is obtained by a dense sweep of 768 Configurations — the grid  $\text{n\_iters} \in \{1, 3, 5\}$ ,  $R_0 \in \{2, 8, 10, 50\}$ ,  $\text{max\_range} \in \{10, 15\}$ ,  $\text{n\_samples} \in \{256, 1024\}$ ,  $\text{fov} \in \{\pi/3, 2\pi\}$ ,

728  $\text{horizon} \in \{2, 5\}$ ,  $\text{period} \in \{1, 4, 8, 10\}$ . We seed LEMCA with 25 of these monoliths spanning the  
 729 entire cost range and run LEMCA up to 20 generations for separate performance targets  $1/\text{cpt}_{90} \in$   
 730  $\{30, 40, 50\}$  (i.e.  $\text{cpt}_{90} \leq \{0.033, 0.025, 0.020\}$ ) under the  $\geq 80\%$  success rate constraint. Every  
 731 design is evaluated over 1000 random starting configurations with a 500-step limit.

## 732 E Points of Comparison

### 733 E.1 B0-MSc: BayesOpt on point-to-disk[CostlyGPS]

734 When candidate evaluations are expensive, black-box optimizers such as Bayesian Optimiza-  
 735 tion (BayesOpt) are a natural tool-kit to consider. BayesOpt, however, can only tune the parameters  
 736 of a *fixed* design template; it cannot alter the program structure (the number of modes, sensor  
 737 types, or transition logic) or otherwise synthesize qualitatively new strategies. To quantify this  
 738 gap, we hand-author four MSC templates that hard-code structural motifs LEMCA synthesizes on  
 739 point-to-disk[CostlyGPS], and use BayesOpt to populate their free parameters (this combination  
 740 is denoted B0-MSc in Figure 4).

741 **Templates.** Each template (Figure 7, bottom; Table 10) instantiates a nested “target-disk” strategy  
 742 through RadialSector detectors: the measured radius  $\|\mathbf{x}\|_2$  partitions the state space into an inner  
 743 mode covering the target disk and one or more outer “approach” modes. The templates progressively  
 744 enrich the radial-band discretization available to the approach mode(s) – supporting finer progressive  
 745 braking as the robot nears the target – while keeping the inner mode minimal. T1 gives the approach  
 746 mode a single radial band; T2 and T3 refine it to 3 and 5 bands; and T4 instead splits the approach  
 747 into two nested modes. For each template, BayesOpt tunes the per-mode angular Sector resolution  
 748  $s_i \in [1, 16]$  and the radial bands  $d_j \in [0, 1]$ , amounting to 3–7 integer/continuous parameters per  
 749 template.

Template	Modes	Free params	Approach-mode radial structure
T1	2	3	single radial band
T2	2	5	3 radial bands
T3	2	7	5 radial bands
T4	3	6	two nested approach modes

Table 10: The four hand-authored B0-MSc templates for point-to-disk[CostlyGPS]. All modes sense via RadialSector detectors and transition based on  $\|\mathbf{x}\|_2$ ; the free parameters are the per-mode Sector resolutions  $s_i$  and the radial band edges  $d_j$  (Figure 7, bottom).

750 **Setup.** We use the Ax/BoTorch BayesOpt client [28], configured to minimize the resource cost  
 751  $R(\mathcal{D})$  subject to the outcome constraint  $J(\mathcal{D}) \geq J_{\text{target}}$  – mirroring LEMCA’s feasible-first objective.  
 752 Crucially, every candidate is evaluated with the *same* MSC-PP0 synthesis procedure LEMCA uses, so  
 753 the two approaches are directly comparable. Starting from the center of the search space, BayesOpt  
 754 iteratively proposes 3 candidates per round for 20 rounds ( $\approx 60$  evaluations per template-target pair,  
 755  $\approx 200$  evaluations in aggregate across templates per target). We sweep three performance targets  
 756  $J_{\text{target}} \in \{200, 300, 450\}$  (the oracle attains  $456.2 \pm 4.6$ ), and for each plot the reciprocal resource  
 757 cost of the best feasible parameterization found so far against the number of evaluations.

758 **Findings.** B0-MSc’s success is highly contingent on both the chosen template and the performance  
 759 target (Figure 7). At the loose target  $J_{\text{target}} = 200$ , only T1 eventually reaches LEMCA’s resource cost,  
 760 and only after  $\approx 48$  evaluations. At the stringent near-oracle target  $J_{\text{target}} = 450$ , the finely-banded T3  
 761 matches LEMCA and T4 marginally exceeds it, but only after  $\approx 30$  evaluations. For the intermediate  
 762 target  $J_{\text{target}} = 300$ , no template reaches LEMCA’s frontier within the budget. In short, even when  
 763 B0-MSc edges out LEMCA, it does so only with (a) a pre-specified template that already encodes the  
 764 right structural prior and (b) runs more evaluations (as it is uninformed + most suitable template is

unknown a-priori). Whereas LEMCA discovers comparable or better designs from scratch by jointly searching over structure and parameters. Combining the complementary strengths of LLM-guided structural search with BayesOpt for fine continuous-parameter tuning is a promising direction for future work.

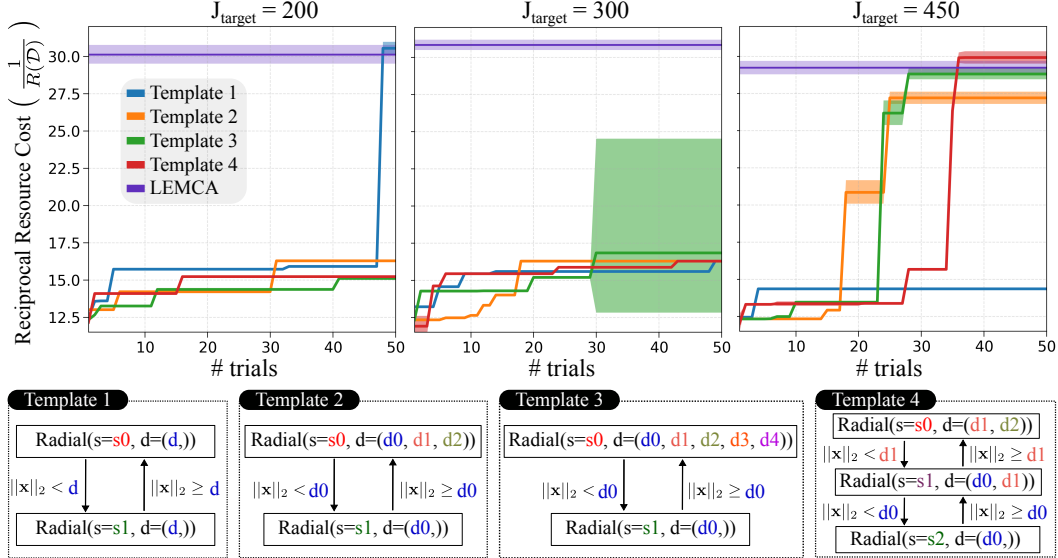


Figure 7: B0-MSO on point-to-disk[CostlyGPS]: we run black-box BayesOpt to populate the free parameters (highlighted in color) of four pre-specified mode-switching templates (T1–T4, bottom; see Table 10). Each panel plots the reciprocal resource cost of the best feasible parameterization found so far against the number of trials (evaluations), for a performance target  $J_{\text{target}} \in \{200, 300, 450\}$ ; the horizontal band is LEMCA’s best design and shaded regions denote  $\pm 1\sigma$  over 1000 rollouts.

## E.2 Alternative LLM-in-the-loop mechanisms

LEMCA wraps the LLM in a genetic algorithm with a population, LLM crossover, feasible-first tournament selection, and a compacted Design Log that summarizes past evaluations as cross-iteration memory (Appendix A.3). To assess which of these ingredients actually drive the search, we ablate the loop into five variants outlined in Table 11 for the clutter-nav task. These variants isolate (i) the genetic algorithm machinery and (ii) the design log. All variants are otherwise identical – same task, seed designs, MPPI controller synthesis, and LLM (gemini-3.1-pro) – so any difference is attributable to the search harness alone.

Variant (legend)	Search mechanism	Pop. + Crossover	Cross-Iter. Memory
GA + Design Log (LEMCA)	feasible-first GA	✓	design log
GA	feasible-first GA	✓	none
Refine Best + Design Log	greedy hill-climb (best)	–	design log
Refine Latest + Design Log	greedy hill-climb (latest)	–	design log
Single Chat	one growing conversation	–	raw chat history

Table 11: The five LLM-in-the-loop variants compared in Figure 8. GA removes only the design log from LEMCA; the two Refine variants replace the population/crossover with greedy single-design refinement (of the global best or the most recent design, respectively) while keeping the design log; Single Chat additionally drops the design log, letting an unbounded conversation history serve as the only memory.

**Setup.** We run the ablation on clutter-nav (Appendix D.4), under the success constraint  $\geq 80\%$  and two reciprocal collision-rate targets  $1/\text{cpt}_{90} \in \{30, 40\}$  (i.e.  $\text{cpt}_{90} \leq \{0.033, 0.025\}$ ). Each variant is run for 3 seeds and up to 20 generations, and we track the reciprocal resource cost of the best feasible design found so far against the generation count (Figure 8; median seed in bold, individual seeds in faint colors).

**Findings.** Two trends emerge in Figure 8. First, the GA harness is the dominant factor: both GA variants – with *and* without the design log – converge to cheaper feasible designs compared to the three single-design variants, which plateau  $\approx 50\text{--}60\%$  higher in resource cost. The population and LLM crossover inject the stochastic diversity needed to escape the local optima that greedy hill-climbing (Refine Best/Refine Latest) and the single growing conversation (Single Chat) settle into early. Second, the effect of the design log is equivocal: removing it from the GA does not hurt – that variant often reaches low-cost designs at least as quickly – whereas among the greedy variants the design log drives steady refinement but can also anchor the search to early suboptimal commitments. We therefore conclude that the GA framework is essential, while the design log’s contribution is setting-dependent and inconclusive.

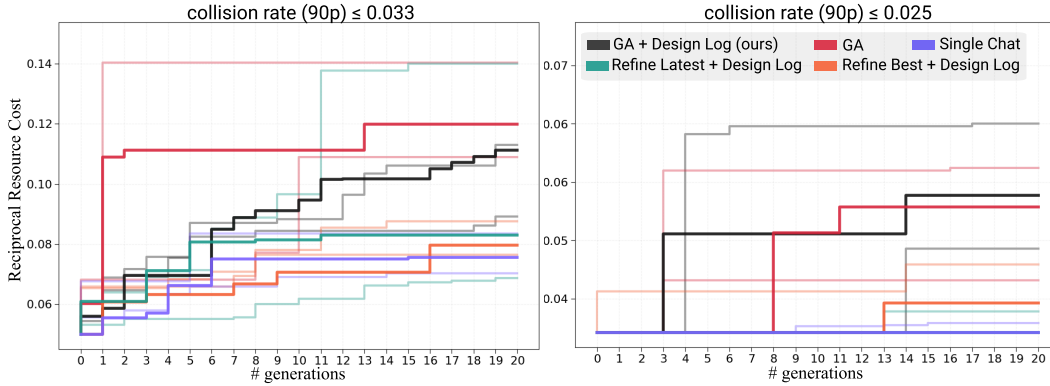


Figure 8: Ablation of LEMCA’s LLM-in-the-loop search mechanism on clutter-nav (Table 11). Each panel plots the reciprocal resource cost of the best feasible design found so far against the generation count for a reciprocal collision-rate target ( $1/\text{cpt}_{90} = 30$ , i.e.  $\text{cpt}_{90} \leq 0.033$ , left; and 40, i.e.  $\leq 0.025$ , right) under the  $\geq 80\%$  success constraint; the median of 3 seeds is represented in bold and individual seeds are marked in faint colors. Both GA variants (GA + Design Log, GA) reach substantially cheaper designs than the greedy single-design (Refine Best/Latest) and Single Chat variants – the GA harness clearly drives improvement, whereas adding the design log to the GA yields no clear advantage.

## F Extended Related Work

**System Co-Design.** Censi [50] presents monotone co-design theory which provides a tool to compositionally reason about resource-functionality trade-offs in system design. While this presents a useful formalism, several components of the design space need to be extensively characterized analytically or empirically [51]. LEMCA can be viewed as a complementary tool in the larger co-design problem to help empirically characterize resource-performance trade-offs in controller synthesis.

**Minimalist Robots.** A classical line of work characterizes a notion of *least* information a robot needs to solve a task or represent its controller, such as: action-based sensors [1, 2], sensor lattices [3, 52], motion description languages [53]. These results give lower bounds and analytic constructions of minimal information sensor/controller specification for specific tasks. LEMCA hinges on similar notions, but relies on the LLM’s potential as a useful idea generator to scalably explore the vast design space and provide constructive evidence of resource-efficient designs on more complex tasks.